PRISM

Professional Remote Infrastructure & Services Management

Easy Infrastructure and Account Management System

Arsitektur Sistem, Technology Stack, Alur Kerja, dan Spesifikasi Service

---

Dokumen Teknis — Versi 3.1

Maret 2026

1. Gambaran Umum Arsitektur PRISM

PRISM mengadopsi arsitektur **hub-and-spoke** di mana Prism Hub bertindak sebagai pusat koordinasi sentral dan setiap Prism Agent di server target bertindak sebagai spoke. Seluruh komunikasi terpusat melalui Hub untuk memudahkan manajemen, monitoring, dan keamanan.

1.1 Komponen Utama

Komponen	Peran	Lokasi
Prism Agent	Lightweight daemon di setiap server target. Service discovery, monitoring, eksekusi perintah.	Setiap server yang dikelola
Prism Hub/Backend	Pusat koordinasi: mengelola koneksi agent, mengagregasi data, REST API, WebSocket gateway.	Server pusat (dedicated)
Prism Web App	Dashboard web: monitoring, approval agent, manajemen service dan akun.	Browser (prism.smartsystem.id)
SQLite Database	Penyimpanan utama: data server, service, akun, telemetry, audit trail.	Co-located dengan Hub (embedded)
Valkey Cache	In-memory cache: session, real-time metrics, agent state, frequently accessed data.	Co-located atau dedicated instance

Agent selalu menginisiasi koneksi ke Hub (**outbound only**). Hub tidak pernah menginisiasi koneksi ke agent, meminimalkan attack surface karena server target tidak perlu membuka port inbound tambahan.

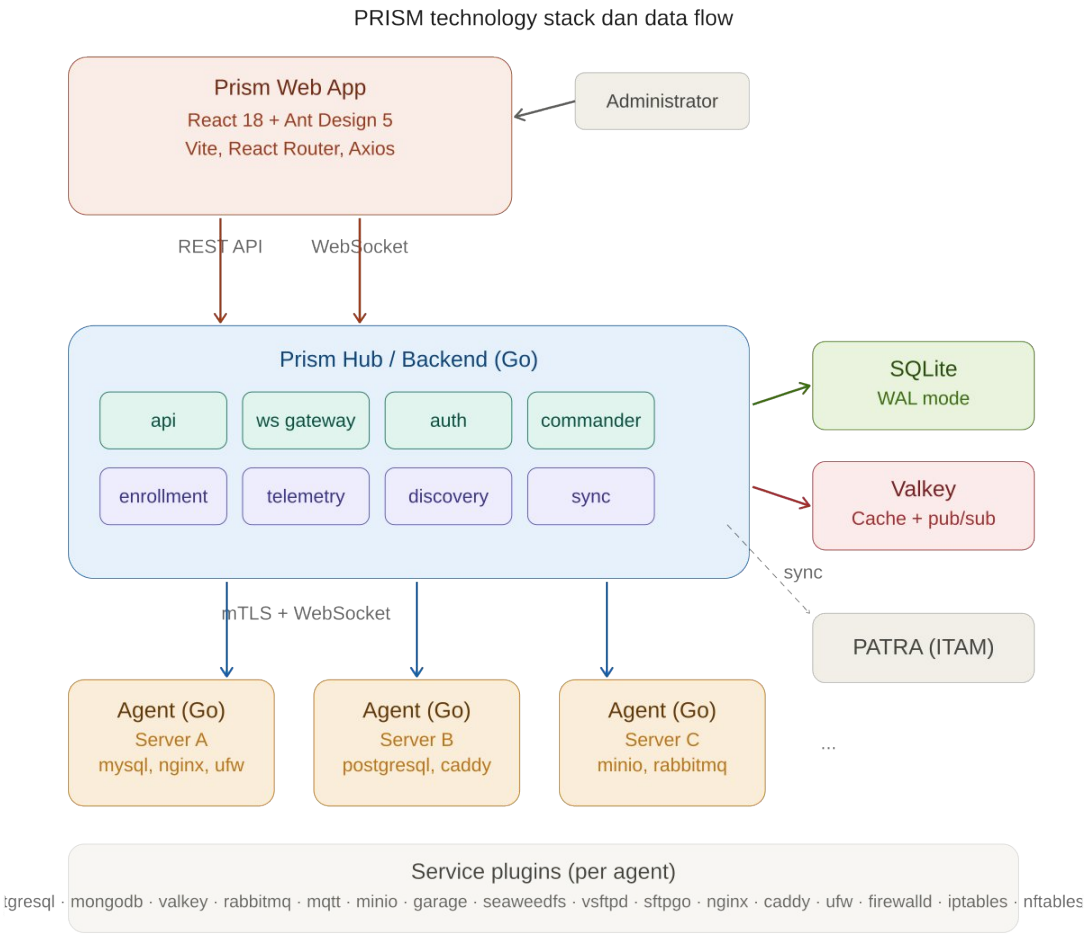
2. Technology Stack

PRISM dibangun dengan technology stack yang dipilih berdasarkan pertimbangan performa, efisiensi resource, kemudahan deployment, dan keandalan.

2.1 Ringkasan Stack

Layer	Teknologi	Justifikasi
Frontend (Web App)	React + Ant Design (AntD)	React memberikan component-based architecture yang scalable. Ant Design menyediakan enterprise-grade UI component library dengan design system yang konsisten, kaya fitur (table, form, modal, notification), dan mendukung theming.
Backend (Hub)	Go (Golang)	Performa tinggi dengan built-in concurrency (goroutines) untuk menangani ribuan WebSocket connections secara efisien. Compiled binary tanpa runtime dependency, memudahkan deployment. Memory footprint rendah.
Agent	Go (Golang)	Binary tunggal yang ringan dan cross-platform. Tidak memerlukan runtime (Python/Node/Java) di server target. Startup cepat, memory usage minimal, native support untuk system calls.
Database	SQLite	Embedded database tanpa server process terpisah, zero-configuration. Ideal untuk single-node deployment. File-based sehingga mudah di-backup. Mendukung WAL mode untuk concurrent read.
Cache	Valkey (Redis-compatible)	In-memory data store untuk caching layer. Mengurangi beban query ke SQLite. Menyimpan real-time state (agent status, metrics, session). Mendukung pub/sub untuk event distribution.

Gambar 1: Arsitektur dan Technology Stack PRISM



2.2 Frontend: React + Ant Design

Prism Web App dibangun menggunakan **React** sebagai UI framework dengan **Ant Design (AntD)** sebagai component library utama. Kombinasi ini memberikan developer experience yang produktif dan user experience yang konsisten.

Arsitektur Frontend

Aspek	Detail
Framework	React 18+ dengan functional components dan hooks
UI Library	Ant Design 5.x — enterprise-grade component library
State Management	React Context + useReducer untuk global state (auth, theme). Local state via useState untuk komponen individual. Opsional: Zustand untuk complex state jika diperlukan.
Routing	React Router v6 dengan lazy loading per route untuk optimasi bundle size

Aspek	Detail
API Communication	Axios atau fetch API untuk REST calls ke Hub. Native WebSocket API untuk real-time data (telemetry, logs, notifications).
Build Tool	Vite — fast HMR untuk development, optimized build untuk production
Styling	Ant Design token-based theming + CSS Modules untuk custom styling
Authentication	JWT token disimpan di httpOnly cookie. Refresh token rotation otomatis.

Pemetaan Komponen Ant Design ke Fitur PRISM

Berikut adalah pemetaan komponen Ant Design yang digunakan untuk setiap fitur utama di Prism Web App:

Fitur PRISM	Komponen AntD Utama	Keterangan
Halaman Servers (list)	Table, Tag, Badge, Button, Input.Search	Tabel server dengan status badge, filter, search, dan tombol aksi
Pending Agent Panel	Alert, Descriptions, Space, Button, Popconfirm	Panel amber/warning untuk agent pending dengan tombol Approve/Reject
Form Akuisisi Server	Modal, Form, Input, Input.TextArea, Button	Modal dialog dengan form nama dan deskripsi server
Server Card	Card, List, Tag, Progress, Statistic, Divider	Card dengan header info, list service, dan quick stats
Service List dalam Card	List.Item, Avatar (icon), Tag (status), Typography.Text	Setiap service ditampilkan sebagai list item dengan icon kategori dan status tag
Modal Konfigurasi Service	Modal (fullscreen/large), Tabs, Descriptions, Table, Form, Switch, Select	Modal besar dengan tab-based layout untuk Overview/Accounts/Configuration/Logs
Tab Overview	Descriptions, Statistic, Row/Col, Progress	Layout grid menampilkan metrik dan informasi service
Tab Accounts	Table (editable), Button, Drawer, Form, Tag, Select	Tabel user/akun dengan inline edit, drawer untuk create/edit, tag untuk kategorisasi
Tab Configuration	Table, Collapse, Form, Switch, InputNumber, Select, Popconfirm	Tabel konfigurasi dengan collapsible sections dan form edit
Tab Logs	Typography.Text (monospace), Input.Search, Select (log level filter)	Live log viewer dengan auto-scroll, search, dan filter level
Aksi Service (Start/Stop/Restart)	Button, Popconfirm, notification, message	Tombol aksi dengan konfirmasi dan feedback notification
Dashboard Overview	Row/Col, Card, Statistic, Chart (via Ant Design Charts)	Dashboard utama dengan KPI cards dan visualisasi

Fitur PRISM	Komponen AntD Utama	Keterangan
User Management / RBAC	Table, Form, Transfer (untuk permission), Tree (untuk role hierarchy)	Manajemen user internal PRISM dengan role assignment
Audit Log	Table (dengan pagination dan filter), DatePicker.RangePicker, Tag	Log audit dengan filter tanggal, user, dan aksi
Notification / Alert	notification, message, Badge (di header), Drawer (notification center)	Sistem notifikasi real-time untuk alert dan event penting

2.3 Backend (Hub): Go

Prism Hub dibangun dengan **Go (Golang)** yang memberikan performa tinggi dan native concurrency untuk menangani ribuan koneksi WebSocket secara simultan.

Arsitektur Backend

Aspek	Detail
Bahasa	Go 1.22+
HTTP Framework	Standard library net/http + gorilla/mux atau chi router. Middleware chain untuk auth, logging, CORS, rate limiting.
WebSocket	gorilla/websocket atau nhooyr.io/websocket untuk koneksi persisten agent. Connection pool dengan health check.
Database Access	mattn/go-sqlite3 (CGO) atau modernc.org/sqlite (pure Go) dengan database/sql interface. Query builder opsional (sqlc atau squirrel).
Cache Client	go-redis/redis (v9) sebagai client library Valkey. Connection pooling built-in.
TLS/mTLS	crypto/tls standard library untuk mTLS handshake. crypto/x509 untuk certificate management (internal CA).
JWT	golang-jwt/jwt (v5) untuk signing dan validasi JWT token (RS256).
Concurrency Model	Goroutine per agent connection. Channel-based communication antar komponen. sync.Pool untuk object reuse.
Logging	Structured logging dengan zerolog atau slog (Go 1.21+ built-in).
Configuration	Environment variables + config file (YAML/TOML). Viper library untuk config management.
Binary Output	Single compiled binary. Tidak memerlukan runtime terpisah. Didistribusikan via container image atau direct binary.

Modul Internal Hub

Hub diorganisir dalam beberapa modul internal yang masing-masing bertanggung jawab atas domain tertentu:

Modul	Tanggung Jawab	Interaksi Utama
api	REST API endpoints untuk Web App. Routing, request validation, response formatting.	Menerima request dari frontend, query ke db/cache, kirim command ke agent via ws
ws (WebSocket Gateway)	Mengelola koneksi WebSocket dari agent. Connection lifecycle, message routing, heartbeat monitoring.	Menerima data dari agent, forward command dari api module
auth	Authentication dan authorization.	Digunakan oleh api dan ws untuk validasi

Modul	Tanggung Jawab	Interaksi Utama
	JWT issuance/validation, mTLS certificate management, RBAC enforcement.	setiap request
enrollment	Proses registrasi agent baru. PSK validation, CSR signing, certificate issuance.	Dipanggil oleh ws saat menerima agent.hello message
discovery	Memproses hasil service discovery dari agent. Service identification, metadata extraction.	Menerima data dari ws, simpan ke db
commander	Membuat dan mendistribusikan command ke agent. Command queue, delivery confirmation, retry logic.	Dipanggil oleh api, dikirim via ws
telemetry	Memproses data telemetry dari agent. Aggregasi, threshold checking, alert triggering.	Menerima dari ws, simpan ke db/cache, trigger notification
cache	Abstraksi layer untuk Valkey. Get/set/invalidate pattern, pub/sub subscription.	Digunakan oleh semua modul untuk caching
db	Database access layer. Migration, query functions, transaction management.	Digunakan oleh semua modul untuk persistence
sync	Sinkronisasi data ke PATRA. Data transformation, API client, retry mechanism.	Membaca dari db, kirim ke Patra Hub API
notifier	Sistem notifikasi. Alert routing, WebSocket push ke frontend, webhook dispatch.	Dipanggil oleh telemetry dan commander saat event penting

2.4 Agent: Go

Prism Agent juga dibangun dengan **Go**, menghasilkan single binary yang ringan dan dapat di-deploy di berbagai platform tanpa dependency tambahan.

Arsitektur Agent

Aspek	Detail
Binary Size	Estimasi 10–20 MB (single static binary, compressed). Tidak memerlukan runtime.
Target Platform	Linux (amd64, arm64). Opsional: FreeBSD, macOS untuk development.
Privilege	Berjalan sebagai systemd service dengan user dedicated (prism-agent). Capability-based permission untuk akses service.
Resource Usage	Target: < 50 MB RAM, < 1% CPU idle. Spike saat discovery atau command execution.
Configuration	File konfigurasi YAML: hub_url, enrollment_key, agent_id (auto-generated), tls cert paths, scan interval.
Plugin Architecture	Setiap jenis service dikelola oleh plugin/module terpisah (mysql_plugin, nginx_plugin, dll.). Plugin register otomatis saat startup berdasarkan service yang terdeteksi.
Offline Mode	Jika koneksi ke Hub terputus, agent tetap mengumpulkan data ke local buffer (SQLite lokal atau in-memory). Flush ke Hub saat koneksi pulih.

Service Plugin Agent

Agent menggunakan arsitektur **plugin** di mana setiap jenis service yang didukung memiliki plugin tersendiri. Plugin bertanggung jawab atas: detection (mengenali apakah service ada di server), collection (mengumpulkan status dan metrik), account management (CRUD operasi akun), dan command execution (start/stop/restart, config update).

Plugin	Detection Method	Data Collection	Account Operations
mysql	Cek process mysqld, port 3306, socket file	SHOW STATUS, SHOW VARIABLES, SHOW DATABASES, SHOW PROCESSLIST	CREATE/ALTER/DROP USER, GRANT/REVOKE
postgresql	Cek process postgres, port 5432, pg_isready	pg_stat_activity, pg_stat_database, pg_roles	CREATE/ALTER/DROP ROLE, GRANT/REVOKE
mongodb	Cek process mongod, port 27017	db.serverStatus(), db.stats(), rs.status()	db.createUser(), db.updateUser(), db.dropUser()
valkey	Cek process valkey-	INFO command (semua	ACL SETUSER, ACL

Plugin	Detection Method	Data Collection	Account Operations
	server, port 6379	section)	DELUSER, ACL LIST
rabbitmq	Cek process beam.smp, management API port 15672	RabbitMQ Management HTTP API	PUT/DELETE /api/users, /api/permissions
mqtt	Cek process mosquitto, port 1883	SYS topics, dynamic security plugin API	Dynamic security plugin: createClient/deleteClient
minio	Cek process minio, S3 port 9000	MinIO Admin API: info, storage info	Admin API: add-user, set-policy, add-service-account
garage	Cek process garage, API port 3900	Admin API: /status, /health	Admin API: key create/delete, bucket allow
seaweedfs	Cek process weed master/filer, port 9333/8888	Master /cluster/status, filer /	S3 IAM via filer config
vsftpd	Cek process vsftpd, port 21, config /etc/vsftpd.conf	Parse config, check PAM/virtual users	Edit user DB (db_load), PAM config
sftpgo	Cek process sftpgo, API port 8080	SFTPGo REST API: /api/v2/status, /api/v2/connections	REST API: /api/v2/users (CRUD)
nginx	Cek process nginx, port 80/443	stub_status module, parse sites-enabled	N/A (config-based, bukan account-based)
caddy	Cek process caddy, admin API port 2019	Admin API: /config/, /reverse_proxy/upstreams	N/A (config-based)
ufw	Cek ufw command available, /etc/ufw/	ufw status verbose, ufw app list	N/A (rule-based)
firewalld	Cek process firewalld, firewall-cmd	firewall-cmd --list-all-zones	N/A (zone/rule-based)
iptables	Cek iptables command, kernel module	iptables -L -n -v (per table)	N/A (rule-based)
nftables	Cek nft command, nftables service	nft list ruleset	N/A (rule-based)
crowdsec	Cek process crowdsec, cscli available	cscli metrics, cscli alerts list, cscli decisions list	N/A (decision/scenario-based)
pm2	Cek process PM2 (pm2 ping), pm2 jlist	pm2 jlist (JSON process list): name, status, CPU, memory, restarts, uptime	N/A (process-based, bukan account-based)
supervisor	Cek process supervisord, supervisorctl	supervisorctl status, supervisorctl avail	N/A (program-based)

Plugin	Detection Method	Data Collection	Account Operations
	available		
systemd_user	Scan systemd user units via systemctl --user (per user context)	systemctl --user list-units, status per unit, journalctl --user-unit	N/A (unit-based, user scope only)
docker	Cek docker daemon (docker info), socket /var/run/docker.sock	docker ps, docker stats, docker inspect, docker images	N/A (container-based). Image pull/remove, network/volume management.
podman	Cek podman binary, podman info (rootless atau rootful)	podman ps, podman stats, podman inspect, podman images	N/A (container-based). Rootless by default. Compatible API dengan Docker.

2.5 Database: SQLite

PRISM menggunakan **SQLite** sebagai database utama. SQLite dipilih karena sifatnya yang embedded (tidak memerlukan server process terpisah), zero-configuration, dan menghasilkan single file yang mudah di-backup dan di-migrate.

Konfigurasi SQLite untuk Production

Parameter	Nilai	Alasan
Journal Mode	WAL (Write-Ahead Logging)	Memungkinkan concurrent read saat write berlangsung. Menghilangkan lock contention antara API read dan telemetry write.
Busy Timeout	5000 ms	Menunggu hingga 5 detik jika database locked, mencegah SQLITE_BUSY error pada concurrent access.
Synchronous	NORMAL	Keseimbangan antara durability dan performa. FULL terlalu lambat untuk telemetry, OFF terlalu berisiko.
Cache Size	-64000 (64 MB)	Memperbesar page cache SQLite untuk mengurangi disk I/O pada query yang sering diakses.
Foreign Keys	ON	Menegakkan referential integrity antar tabel.
Auto Vacuum	INCREMENTAL	Mengoptimalkan ukuran file database secara bertahap tanpa blocking.

Skema Tabel Utama

Berikut adalah tabel-tabel utama dalam database PRISM:

Tabel	Primary Key	Kolom Utama	Relasi
servers	id (TEXT, UUID)	name, description, hostname, ip_address, os, status (pending/active/unreachable/removed), agent_version, created_at, updated_at	Has many: services, accounts
services	id (TEXT, UUID)	server_id (FK), name, type, version, port, status (running/stopped/error), config_path, pid, discovered_at, updated_at	Belongs to: servers. Has many: accounts
accounts	id (TEXT, UUID)	service_id (FK), server_id (FK), username, account_type (user/admin/service), category (project/independent), project_name, permissions (JSON), status (active/disabled), created_at	Belongs to: services, servers

Tabel	Primary Key	Kolom Utama	Relasi
agent_certificates	id (TEXT, UUID)	server_id (FK), fingerprint, issued_at, expires_at, revoked_at, revocation_reason	Belongs to: servers
enrollment_keys	id (TEXT, UUID)	key_hash, created_by, expires_at, used_at, used_by_server_id	Used by: servers
commands	id (TEXT, UUID)	server_id (FK), service_id (FK), type, payload (JSON), status (pending/sent/executed/failed), issued_by, issued_at, executed_at, result (JSON)	Belongs to: servers, services
telemetry	id (INTEGER, auto)	server_id (FK), service_id (FK), metrics (JSON), collected_at	Belongs to: servers, services. Partitioned by time.
audit_log	id (INTEGER, auto)	user_id, action, resource_type, resource_id, details (JSON), ip_address, timestamp	Standalone audit trail
users	id (TEXT, UUID)	username, email, password_hash, role (admin/operator/viewer), mfa_enabled, last_login, status	PRISM internal users (bukan service accounts)

Catatan tentang Skalabilitas SQLite

SQLite dengan WAL mode mampu menangani ratusan server dan ribuan service dengan baik. Untuk deployment yang lebih besar (1000+ server), tabel telemetry dapat dirotasi menggunakan time-based partitioning (tabel per hari/minggu) atau di-migrate ke time-series database terpisah. Data yang sering diakses di-cache di Valkey untuk mengurangi beban query.

2.6 Cache: Valkey

Valkey (fork Redis yang fully open-source) digunakan sebagai caching layer di PRISM. Valkey menyimpan data yang sering diakses dan data real-time yang berubah cepat, mengurangi beban query ke SQLite dan meningkatkan responsiveness Web App.

Strategi Caching

Data	Key Pattern	TTL	Strategi
Agent online status	agent:{server_id}:status	60 detik (diperbarui tiap heartbeat)	Write-through: setiap heartbeat update Valkey. Jika key expired = agent unreachable.
Service status per server	server:{server_id}:services	30 detik	Write-through: diperbarui saat telemetry masuk. Frontend membaca dari cache.
Real-time metrics	metrics:{server_id}:{service_id}	60 detik	Write-through: data CPU, memory, connections. Dioverwrite setiap telemetry push.
Server list (active)	servers:active:list	5 menit	Cache-aside: populate saat pertama kali diakses. Invalidate saat ada perubahan server.
JWT refresh token mapping	auth:refresh:{token_hash}	24 jam	Write-through: disimpan saat token issued. Dihapus saat revoke atau rotation.
User session	session:{user_id}	Sesuai session config	Write-through: menyimpan data session user Web App.
Command status	cmd:{command_id}:status	1 jam	Write-through: update real-time saat command dispatched/executed/failed.
Rate limiting counters	ratelimit:{ip}:{endpoint}	1 menit (sliding window)	Increment on request: menghitung request count per IP per endpoint.
Dashboard aggregate stats	dashboard:stats	30 detik	Cache-aside: aggregate query di-cache. Invalidate periodik.

Pub/Sub untuk Real-time Events

Valkey Pub/Sub digunakan untuk mendistribusikan event real-time dari Hub ke frontend (via WebSocket) tanpa polling:

Channel	Publisher	Subscriber	Event
events:server:{id}	ws module (saat menerima data agent)	API module (forward ke frontend WebSocket)	Agent connect/disconnect, status change, new telemetry
events:service:{id}	ws module, commander module	API module	Service start/stop/error, config change, account change
events:alerts	telemetry module	API module, notifier module	Threshold breach, agent unreachable, certificate expiring
events:commands:{id}	commander module	API module	Command dispatched, executed, failed (untuk progress tracking di frontend)

2.7 Data Flow: Bagaimana Komponen Berinteraksi

Berikut adalah alur data untuk operasi-operasi utama di PRISM, menunjukkan bagaimana setiap komponen teknologi berinteraksi:

Read Path (Admin membuka dashboard)

1. **Browser (React)** mengirim GET request ke Hub REST API.
2. **Hub (Go)** memeriksa JWT token di request header untuk autentikasi.
3. **Hub** mengecek Valkey cache terlebih dahulu untuk data yang diminta.
4. **Cache hit:** data langsung dikembalikan ke frontend. Cache miss: Hub query ke SQLite, hasilnya disimpan ke Valkey, lalu dikembalikan.
5. **Frontend (React)** merender data menggunakan komponen Ant Design (Table, Card, Statistic).

Write Path (Admin mengirim command ke service)

1. **Frontend** mengirim POST request ke Hub REST API dengan command payload.
2. **Hub (api module)** memvalidasi request, menyimpan command ke SQLite dengan status “pending”.
3. **Hub (commander module)** mengirim command ke agent via WebSocket, update status ke “sent” di Valkey.
4. **Hub** publish event ke Valkey Pub/Sub channel events:commands:{id}.
5. **Agent (Go)** menerima command, mengeksekusi (misalnya systemctl restart mysql), mengirim result kembali via WebSocket.
6. **Hub (ws module)** menerima result, update status command di SQLite dan Valkey, publish event.
7. **Frontend** menerima event via WebSocket, menampilkan notification (AntD message/notification).

Telemetry Path (Agent mengirim data periodik)

1. **Agent (Go)** mengumpulkan metrik dari service plugins setiap 60 detik.
2. **Agent** mengirim data via WebSocket ke Hub.
3. **Hub (ws module)** menerima data, forward ke telemetry module.
4. **Hub (telemetry module)** memproses data: update Valkey cache (real-time metrics), insert ke SQLite (historical), cek threshold untuk alerting.
5. **Hub** jika ada threshold breach, publish alert ke Valkey Pub/Sub, trigger notifier module.
6. **Frontend** menerima update via WebSocket subscription, re-render komponen Statistic/Progress secara real-time.

3. Alur Kerja PRISM

Berikut adalah urutan proses kerja lengkap dari saat agent pertama kali melapor hingga administrator dapat mengelola service secara penuh.

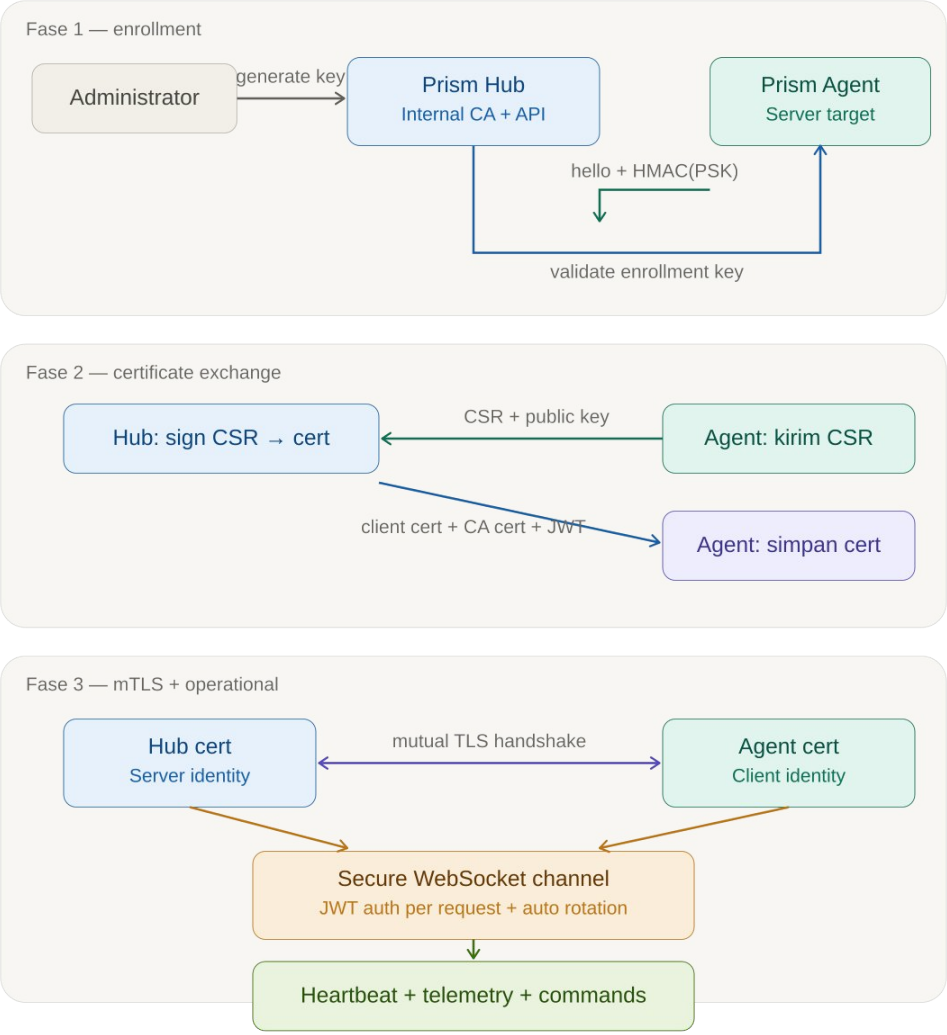
3.1 Fase 1: Agent Melapor ke Hub (Otomatis)

Ketika Prism Agent pertama kali dijalankan, agent secara otomatis menginisiasi koneksi ke Hub untuk mendaftarkan diri. Proses ini berjalan **tanpa intervensi administrator**.

1. **Agent Startup:** Agent binary dijalankan di server target. Konfigurasi awal berisi Hub URL dan Pre-Shared Enrollment Key (PSK) yang bersifat one-time-use.
2. **Hello Message:** Agent mengirim pesan pertama ke Hub melalui TLS 1.3 berisi: HMAC-SHA256 dari PSK + timestamp, Certificate Signing Request (CSR) berisi public key agent, dan metadata server (hostname, OS, IP, agent version).
3. **Hub Validates:** Hub memverifikasi HMAC terhadap PSK yang tersimpan. Jika valid dan PSK belum expired/terpakai, Hub menandai agent sebagai “pending approval”.
4. **Certificate Issuance:** Hub (sebagai internal CA) menandatangani CSR dan mengirim kembali: signed client certificate (X.509), CA certificate, dan initial JWT access + refresh token.
5. **mTLS Established:** Agent membentuk koneksi baru menggunakan mutual TLS. Kedua pihak saling memverifikasi certificate. Enrollment key diinvalidasi.
6. **Service Discovery:** Agent melakukan scanning otomatis (systemd units, docker containers, process list) dan melaporkan hasilnya ke Hub.
7. **Status: Pending Approval.** Agent terhubung aman dengan status “pending approval”, menunggu administrator.

Gambar 2: Alur Registrasi Agent dan Token Exchange

Alur registrasi agent dan token exchange di PRISM



3.2 Fase 2: Admin Melihat dan Meng-approve Agent

Administrator membuka Prism Web App dan navigasi ke **tab Servers**.

Pending Agent di Frontend

Agent berstatus “pending approval” ditampilkan dalam panel terpisah (AntD **Alert** component dengan type “warning”) dengan informasi:

Informasi	Komponen AntD	Contoh
Hostname	Descriptions.Item	srv-prod-db-01
IP Address	Descriptions.Item	10.0.1.45
OS	Descriptions.Item + Tag	Ubuntu 24.04 LTS
Agent Version	Tag	v1.2.0
Detected Services	Badge (count)	5 services
Connected Since	Descriptions.Item	2026-03-19 14:32 WIB
Status	Tag (color: gold)	Pending Approval
Aksi	Button (primary) + Popconfirm (danger)	[Approve] [Reject]

Proses Approval

1. **Admin menekan tombol “Approve”.** Muncul AntD Modal dengan Form di dalamnya.
2. **Admin mengisi form:** Server Name (Input, required) dan Description (Input.TextArea, optional).
3. **Admin menekan “Save” / “Acquire”.** Frontend mengirim POST ke Hub API. Hub mengubah status agent dari “pending” ke “active”, menyimpan nama dan deskripsi, menginstruksikan agent memulai full monitoring.
4. **Feedback:** AntD notification (type: success) muncul. Agent berpindah dari panel pending ke daftar server aktif.

3.3 Fase 3: Server Card Tampil

Setelah approval, server muncul sebagai **Card** (AntD Card component) di halaman Servers. Card ini me-list semua service yang terdeteksi oleh agent.

Anatomi Server Card

Bagian Card	Komponen AntD	Isi
Card Header	Card title + extra	Server Name, Status Badge (running=green, warning=amber, error=red), IP Address
Metadata Row	Space + Tag	OS tag, uptime, agent version, last heartbeat (Tooltip: exact timestamp)
Quick Stats	Row + Col + Statistic	Total: 5 | Running: 4 | Stopped: 0 | Error: 1
Service List	List + List.Item	Setiap service: icon kategori (Avatar), nama + versi (Text), status (Tag)
Footer	Card actions	Edit Server, Restart Agent, Remove (Popconfirm)

Kategori Service

Kategori	Service	Tag Color (AntD)
Database	MySQL, PostgreSQL, MongoDB, Valkey	green
Message Broker	RabbitMQ, MQTT (Mosquitto)	purple
Object Storage	MinIO, Garage, SeaweedFS	blue
File Transfer	vsftpd, SFTPGo	cyan
Web Server / Reverse Proxy	Nginx, Caddy	orange
Firewall & Security	UFW, firewalld, iptables, nftables, CrowdSec	red
Process Supervisor	PM2, Supervisor, systemd user services	geekblue
Container Runtime	Docker, Podman	magenta

3.4 Fase 4: Modal Konfigurasi Service

Ketika admin mengklik service dari Server Card, muncul **Modal besar** (AntD Modal width: 900px+) dengan **Tabs** layout berisi 4 tab: Overview, Accounts, Configuration, dan Logs.

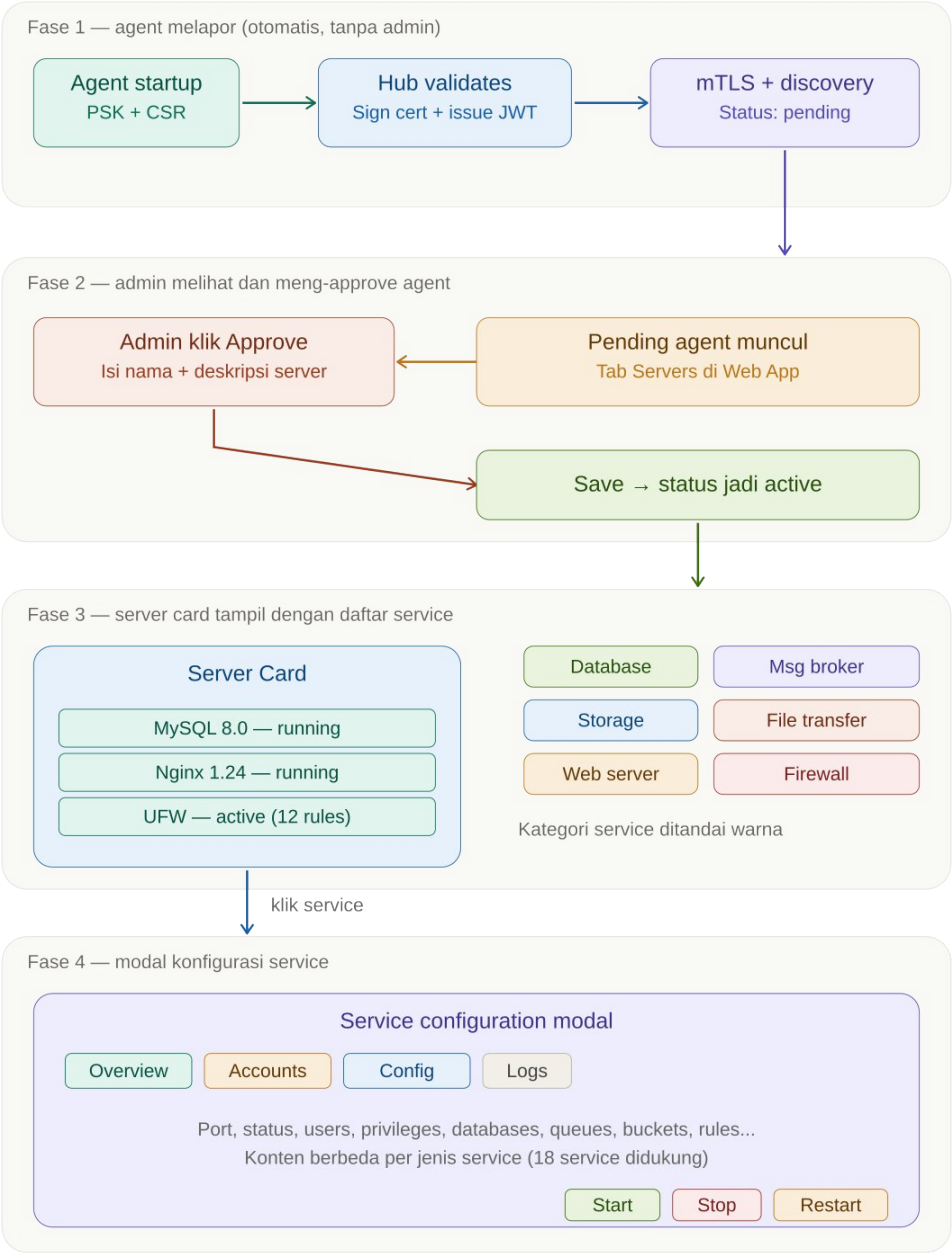
Struktur Umum Modal

Tab	Komponen AntD Utama	Isi
Header (di atas tabs)	Space, Tag, Typography.Title, Statistic	Nama service, versi, status badge, uptime, PID
Overview	Descriptions, Row/Col, Statistic, Progress	Port, config path, memory, CPU, connections, service-specific metrics
Accounts	Table (sortable, filterable), Button, Drawer, Form, Tag	Daftar user/akun + CRUD. Kategorisasi project-based / independent.
Configuration	Descriptions, Table, Collapse, Form, Switch, Select	Field konfigurasi spesifik per service type
Logs	div (monospace, auto-scroll), Input.Search, Select	Live tail log via WebSocket stream
Footer	Space, Button, Popconfirm	Start / Stop / Restart / Apply Config / Refresh

Detail field konfigurasi per service type tersedia di **Lampiran A** (di bawah). Setiap service memiliki field Overview, Accounts, dan Configuration yang berbeda sesuai kemampuan service tersebut.

Gambar 3: Alur Kerja dari Agent Melapor hingga Manajemen Service

Alur kerja PRISM: dari agent melapor hingga manajemen service



4. Manajemen Projects dan Accounts

PRISM mengelola seluruh akun service ke dalam dua kategori utama: **project-based accounts** dan **independent accounts**. Untuk mendukung hal ini, Prism Web App menyediakan dua menu/tab dedicated di navigasi utama: **Projects** dan **Accounts**.

4.1 Konsep: Project sebagai Resource Group

Dalam PRISM, sebuah **project bukan sekadar label atau tag**. Project adalah **resource group** — kumpulan seluruh resource infrastruktur yang dibutuhkan sebuah aplikasi atau sistem untuk berjalan. Sebuah project biasanya terdiri dari kombinasi beberapa komponen berikut:

Komponen	Fungsi dalam Project	Service di PRISM	Contoh Resource
Database	Menyimpan data aplikasi: user data, transaksi, konten, state.	MySQL, PostgreSQL, MongoDB, Valkey	Database + user + permission khusus untuk project ini
Message Broker	Komunikasi antar microservice, event-driven architecture, task queue.	RabbitMQ, MQTT	Virtual host, exchange, queue, user + permission untuk project ini
Object Storage	Menyimpan file upload, media, backup, static assets.	MinIO, Garage, SeaweedFS	Bucket + access key + policy khusus project ini
Worker / Helper	Background job processing, scheduled tasks, data transformation.	PM2, Supervisor, systemd user	Process: email-worker, pdf-generator, cron-scheduler
AI Worker	Machine learning inference, NLP processing, image recognition.	PM2, Supervisor, systemd user	Process: ai-inference-api, embedding-worker, llm-gateway
Data Pipeline Worker	ETL, data sync, log aggregation, stream processing.	PM2, Supervisor, systemd user	Process: etl-pipeline, kafka-consumer, log-processor
Reverse Proxy / Web Server	Routing traffic ke aplikasi, SSL termination, load balancing.	Nginx, Caddy	Site config / server block yang mengarahkan domain ke aplikasi project
File Transfer	Upload/download file untuk integrasi dengan pihak ketiga.	vsftpd, SFTPGo	User FTP/SFTP + folder + quota khusus project ini
Firewall Rules	Membuka port atau allow IP yang	UFW, firewalld, iptables, nftables	Rules yang spesifik untuk kebutuhan network project ini

Komponen	Fungsi dalam Project	Service di PRISM	Contoh Resource
	dibutuhkan project.		
Security	Proteksi terhadap brute force dan abuse.	CrowdSec	Scenario dan bouncer config yang relevan dengan project ini
Container	Menjalankan service atau aplikasi dalam container yang terisolasi.	Docker, Podman	Container instances, images, volumes, networks khusus project ini

Contoh: Anatomi Project "E-Commerce Platform"

Berikut adalah contoh resource group untuk project e-commerce yang tersebar di beberapa server:

Resource	Server	Service	Detail
PostgreSQL DB user	srv-db-01	PostgreSQL	User: ecommerce_app, DB: ecommerce_prod, Permission: ALL on ecommerce_prod
Valkey namespace	srv-cache-01	Valkey	ACL user: ecommerce, key pattern: ecom:*, permission: +@all ~ecom:*
RabbitMQ vhost	srv-mq-01	RabbitMQ	VHost: /ecommerce, User: ecom_service, Queues: order.created, payment.processed, shipment.updated
MinIO bucket	srv-storage-01	MinIO	Bucket: ecommerce-assets, Access key: ecom-s3-key, Policy: readwrite on ecommerce-assets/*
API server	srv-app-01	PM2	Process: ecom-api (cluster mode, 4 instances), script: /app/ecommerce/api/server.js
Order worker	srv-app-01	PM2	Process: ecom-order-worker, script: /app/ecommerce/workers/order.js
AI recommendation	srv-ai-01	systemd user	Unit: ecom-recommender.service, ExecStart: /app/ecommerce/ai/recommend-server
ETL pipeline	srv-data-01	Supervisor	Program: ecom-etl-sync, command: python /app/ecommerce/etl/sync.py
Web server	srv-web-01	Nginx	Site: ecommerce.example.com → upstream srv-app-01:3000
Firewall	srv-web-01	UFW	Allow 443/tcp from any, Allow 3000/tcp from srv-web-01 only
Redis container	srv-app-01	Docker	Container: ecom-session-redis (redis:7-

Resource	Server	Service	Detail
			alpine), port 6380, volume: ecom-redis-data, network: ecom-internal

Semua resource di atas terkait satu sama lain dan memiliki lifecycle yang sama. Ketika project e-commerce selesai atau di-decommission, **seluruh resource ini perlu di-review dan di-cleanup secara bersamaan**. Inilah value utama dari model project di PRISM.

4.2 Kategorisasi Resource: Accounts dan Processes

Kategorisasi project-based vs independent berlaku untuk **dua jenis resource** yang dikelola PRISM: **accounts** (akun service) dan **processes** (proses custom di PM2/Supervisor/systemd user). Kedua jenis resource ini mengikuti pola kategorisasi yang sama.

Accounts (Akun Service)

Kategori	Deskripsi	Contoh
Project-based	Akun yang terikat langsung pada suatu project. Lifecycle mengikuti lifecycle project.	MySQL user untuk app e-commerce, MinIO access key untuk storage project, RabbitMQ vhost untuk microservices project X
Independent	Akun yang tidak terikat pada project. Bersifat long-lived untuk operasional, monitoring, atau shared access.	Admin global database, akun monitoring Grafana, akun backup scheduler, shared S3 key untuk cross-project assets

Processes (Proses Custom)

Kategori	Deskripsi	Contoh
Project-based	Proses yang merupakan bagian dari stack sebuah project. Lifecycle mengikuti lifecycle project.	API server project X (PM2), order-worker project X (Supervisor), AI inference service project Y (systemd user)
Independent	Proses yang berjalan untuk keperluan infrastruktur umum, maintenance, atau lintas project. Tidak terikat pada project tertentu.	Lihat tabel di bawah

Contoh Independent Processes

Proses-proses independent biasanya menjalankan tugas operasional dan maintenance yang melayani infrastruktur secara keseluruhan, bukan project tertentu:

Proses	Supervisor	Fungsi	Contoh Detail
Storage Cleanup	PM2 / systemd user	Menghapus file lama, temporary file, atau file yang sudah tidak dibutuhkan pada folder target tertentu.	Cron-style process yang scan /data/uploads/ dan hapus file > 30 hari. Atau cleanup /tmp dan /var/cache secara periodik.
Certificate Sync	PM2 / systemd user	Mengupload atau mendownload SSL/TLS certificate dari satu server ke server lain, memastikan semua server menggunakan certificate terbaru.	Process yang watch certificate directory, dan saat ada perubahan, sync ke server Nginx/Caddy lain via SCP/rsync atau API.
Log Rotator / Archiver	Supervisor / systemd user	Mengkompresi, memindahkan, atau menghapus log lama yang tidak ditangani oleh logrotate standar.	Compress log > 7 hari ke .gz, pindahkan ke cold storage (MinIO), hapus lokal > 30 hari.
Database Backup Runner	PM2 / Supervisor	Menjalankan backup database terjadwal dan mengupload hasil backup ke object storage.	Dump PostgreSQL setiap jam, encrypt, upload ke MinIO bucket “db-backups”, hapus dump lokal.
Health Check Monitor	PM2 / systemd user	Memonitor endpoint kesehatan service tertentu dan mengirim alert jika ada masalah (complement untuk monitoring yang lebih besar).	Cek HTTP 200 pada endpoint /health setiap 30 detik. Kirim webhook ke Slack jika gagal.
Metrics Exporter	systemd user	Custom exporter yang mengumpulkan metrik dari sumber non-standar dan expose ke format yang bisa di-scrape (Prometheus, dll.).	Baca status dari hardware sensor / IoT gateway, expose sebagai /metrics endpoint.
Data Sync / Replicator	Supervisor / PM2	Menyinkronkan data antar sistem: database ke data warehouse, file antar server, atau config antar environment.	Sync tabel tertentu dari production DB ke analytics DB setiap 15 menit.
Queue Consumer (shared)	PM2 / Supervisor	Consumer yang memproses pesan dari message broker untuk keperluan lintas project (logging, notification routing, audit trail).	Consume dari exchange “audit.events”, tulis ke audit log database. Melayani semua project.

Proses	Supervisor	Fungsi	Contoh Detail
DNS/Domain Updater	systemd user	Memperbarui DNS record atau dynamic DNS secara periodik.	Update Cloudflare DNS record dengan IP terbaru server jika menggunakan dynamic IP.
Security Scanner	Supervisor / systemd user	Menjalankan vulnerability scan, file integrity check, atau malware scan secara terjadwal.	Scan /var/www dan /app setiap malam, lapor ke admin jika ditemukan file mencurigakan.

Independent Process ≠ Tidak Terpantau

Meskipun tidak terikat project, independent processes tetap sepenuhnya terpantau oleh PRISM: status (running/stopped/error), CPU/memory usage, restart count, dan log. Perbedaannya hanya pada kategorisasi dan lifecycle — independent processes tidak terpengaruh ketika suatu project di-decommission.

4.3 Menu Projects

Menu **Projects** menampilkan seluruh project beserta resource infrastructure yang terasosiasi. Setiap project adalah pandangan lengkap terhadap seluruh akun, proses, dan konfigurasi yang dibutuhkan aplikasi/sistem tersebut.

Halaman Project List

Elemen	Komponen AntD	Keterangan
Project Table / Card Grid	Table atau Card + List	Daftar project dengan mode tampilan table (dense) atau card grid (visual). Toggle via segmented control.
Kolom: Project Name	Typography.Link	Nama project, clickable untuk masuk ke detail project
Kolom: Description	Typography.Text	Deskripsi singkat project
Kolom: Owner / Team	Tag	Nama team atau person yang memiliki project
Kolom: Resources	Badge (multi)	Breakdown resource: 3 accounts, 2 processes, 1 site config — ditampilkan sebagai badges
Kolom: Servers	Badge	Jumlah server yang memiliki resource project ini
Kolom: Status	Tag (active/completed/archived)	Status lifecycle project
Kolom: Created	Typography.Text	Tanggal project dibuat
Aksi	Button (View, Edit, Archive)	Aksi per project

Elemen	Komponen AntD	Keterangan
Tombol: + New Project	Button (primary)	Membuat project baru
Filter & Search	Input.Search, Select (status, owner, server)	Pencarian dan filtering

Halaman Project Detail

Halaman detail project menampilkan **resource map** lengkap — semua akun, proses, dan konfigurasi yang terkait project, dikelompokkan agar mudah dipahami:

Section	Komponen AntD	Isi
Project Info	Descriptions	Nama, deskripsi, owner/team, status, created/updated date
Resource Summary	Row + Col + Statistic	Ringkasan: 4 database accounts, 2 storage keys, 1 broker vhost, 3 worker processes, 1 web config. Total across N servers.
Tab: Accounts	Table (groupable by server/service)	Semua akun project: username, service, server, status, permissions. CRUD dari sini.
Tab: Processes	Table (groupable by server/supervisor)	Semua custom process project: PM2 processes, Supervisor programs, systemd units. Start/stop dari sini.
Tab: Containers	Table (groupable by server/runtime)	Semua container project: Docker/Podman containers, images, status, port mapping, volume mounts. Start/stop/restart dari sini.
Tab: Web / Proxy Config	Table	Site configs di Nginx/Caddy yang terkait project ini: domain, upstream, SSL status.
Tab: Firewall Rules	Table	Firewall rules yang di-tag untuk project ini (jika applicable).
Tab: Resource Map	Collapse / Tree atau Visualisasi graf	Visualisasi hubungan antar resource: server → service → akun/proses. Menunjukkan dependency dan distribusi.
Quick Actions	Button, Popconfirm	Disable All Accounts, Stop All Processes, Export Resource List, Decommission Project
Audit Trail	Table (timeline)	History perubahan: resource ditambahkan/dihapus, permission diubah, process started/stopped.

Project Lifecycle

PRISM mendukung lifecycle management untuk project:

Status	Arti	Dampak pada Resource
Active	Project sedang berjalan. Resource aktif digunakan.	Semua resource berstatus normal. Monitoring dan alerting aktif.
Completed	Project selesai. Resource tidak lagi dibutuhkan.	PRISM menampilkan summary resource dan menawarkan opsi: disable all accounts, stop all processes, atau keep with warning. Resource yang belum di-cleanup mendapat label “orphan”.
Archived	Project di-archive untuk referensi historis.	Seluruh resource seharusnya sudah di-disable/delete. Jika masih ada yang aktif, PRISM menampilkan alert.

4.4 Menu Accounts

Menu **Accounts** menampilkan **unified view** seluruh akun dari semua service di semua server yang dikelola PRISM. Menu ini fokus pada **akun service** (database users, access keys, broker users, FTP users, dll.). Untuk processes (PM2/Supervisor/systemd user), pengelolaan dilakukan melalui tab Processes di halaman Project Detail atau melalui modal konfigurasi process supervisor di Server Card.

Halaman Account List

Elemen	Komponen AntD	Keterangan
Account Table	Table (with pagination, sorting, multi-column filter)	Tabel utama yang menampilkan seluruh akun dari seluruh server
Kolom: Username	Typography.Text	Username atau key ID akun
Kolom: Service	Tag (color per kategori)	Nama service dan tipe (MySQL, MinIO, RabbitMQ, dll.)
Kolom: Server	Typography.Link	Nama server di mana akun berada, clickable ke server detail
Kolom: Category	Tag (blue=project, gray=independent)	Kategorisasi: project-based atau independent
Kolom: Project	Typography.Link atau "—"	Nama project (jika project-based) atau dash (jika independent)
Kolom: Type	Tag	Tipe akun: user, admin, service account
Kolom: Status	Badge (green=active, gray=disabled)	Status akun
Kolom: Permissions	Tooltip + Tag	Ringkasan permission (hover untuk detail)
Kolom: Last Activity	Typography.Text	Last login atau last used timestamp
Filter Panel	Form (inline), Select, DatePicker	Filter by: server, service type, category, project, status, date range
Bulk Actions	Button + Checkbox selection	Disable selected, change category, assign to project, export
Tombol: + New Account	Button (primary)	Membuat akun baru (pilih server → service → isi detail)

Fitur Cross-Reference

Halaman Accounts menyediakan kemampuan cross-reference yang kuat:

Fitur	Deskripsi
Search Global	Mencari akun berdasarkan username di seluruh server dan service. Berguna

Fitur	Deskripsi
	untuk audit: “di mana saja user X memiliki akun?”
Filter by Project	Menampilkan hanya akun yang terkait project tertentu, lintas server dan service.
Orphan Detection	Mendeteksi akun yang masih project-based tetapi project-nya sudah berstatus completed/archived. Alert untuk cleanup.
Duplicate Detection	Mendeteksi username yang sama di beberapa service/server. Berguna untuk standarisasi naming convention.
Compliance View	Menampilkan akun yang belum di-review dalam X hari, akun tanpa MFA (jika applicable), akun dengan excessive permissions.
Export	Export daftar akun ke CSV/Excel untuk keperluan audit external.

4.5 Interaksi Projects dan Accounts dengan Service Modal

Menu Projects dan Accounts terintegrasi erat dengan modal konfigurasi service di Server Card:

Aksi	Alur
Buat akun dari Service Modal	Admin klik service di Server Card → Tab Accounts → + New Account → Isi username, permissions → Pilih kategori (project/independent) → Jika project: pilih project dari dropdown → Save. Akun otomatis muncul di menu Accounts dan di halaman Project detail.
Buat akun dari menu Accounts	Admin klik + New Account di halaman Accounts → Pilih server → Pilih service → Isi detail + kategori → Save. Akun otomatis muncul di Tab Accounts pada service modal yang relevan.
Assign akun ke project	Admin bisa mengubah kategori akun dari independent ke project-based (atau sebaliknya) baik dari Service Modal maupun dari menu Accounts. Perubahan tersinkronisasi secara real-time.
Project selesai	Admin mengubah status project ke “completed” di menu Projects → PRISM menampilkan daftar semua akun terkait → Admin dapat memilih: disable all, delete all, atau keep (dengan warning).

5. Custom Services: Process Supervisors dan Container Runtime

Selain service-service yang sudah dikenal (database, message broker, storage, dll.), PRISM juga mendukung pengelolaan **custom service dan process** melalui tiga jenis process supervisor (**PM2**, **Supervisor**, **systemd user services**) dan dua container runtime (**Docker**, **Podman**). Ini memungkinkan administrator mengelola aplikasi custom, microservices, background workers, containerized services, dan proses lainnya.

5.1 Jenis Process Supervisor yang Didukung

Supervisor	Bahasa/Runtime Target	Kelebihan	Scope
PM2	Node.js (utama), Python, Go, binary apapun	Cluster mode, auto-restart, log management, startup script, ecosystem file. Sangat populer untuk aplikasi Node.js.	User-level process
Supervisor (supervisord)	Bahasa/runtime apapun	Mature, stabil, konfigurasi INI-style sederhana. Mendukung process groups, event listeners, XML-RPC API.	System-level atau user-level
systemd user services	Bahasa/runtime apapun	Terintegrasi dengan OS. Tidak memerlukan software tambahan. Mendukung dependencies, resource limits, sandboxing.	User-level (systemctl --user). Bukan system-wide.

Penting: systemd User Services, Bukan System-Wide

PRISM hanya mengelola systemd services yang berjalan di scope user (systemctl --user), bukan system-wide services. Ini karena: (1) system-wide services dikelola oleh plugin khusus PRISM (mysql, nginx, dll.), (2) user services memberikan isolasi yang lebih baik — setiap user memiliki unit files sendiri di ~/.config/systemd/user/, (3) tidak memerlukan root access untuk start/stop/restart, lebih aman untuk operasi remote, (4) mendukung lingering (loginctl enable-linger) agar service tetap berjalan meskipun user tidak logged in.

5.2 Bagaimana Agent Mendeteksi Process Supervisors

Prism Agent memiliki plugin khusus untuk setiap process supervisor. Saat service discovery, agent memeriksa keberadaan supervisor dan mendaftarkan proses-proses yang dikelolanya:

Supervisor	Detection	Process Enumeration
PM2	Cek apakah PM2 daemon berjalan (pm2 ping). Cek PM2 home directory (~/.pm2).	pm2 jlist menghasilkan JSON array seluruh process: id, name, status, mode (fork/cluster), instances, CPU, memory, restarts, uptime, script path, log paths.
Supervisor	Cek process supervisord running. Cek config /etc/supervisor/ atau custom path.	supervisorctl status menghasilkan list program: name, state (RUNNING/STOPPED/FATAL), PID, uptime. XML-RPC API untuk data lebih detail.
systemd user	Cek systemd user instance aktif untuk setiap user yang di-configure. Gunakan systemctl --user --machine=<user>@ untuk enumerasi.	systemctl --user list-units --type=service menghasilkan list unit: name, load state, active state, sub state, description. Hanya unit yang di-enable atau running yang dilaporkan.

5.3 Container Runtime: Docker dan Podman

PRISM juga mengelola container yang berjalan melalui **Docker** atau **Podman**. Container mengikuti pola kategorisasi yang sama: setiap container (atau Compose project / Podman pod) dapat di-tag sebagai **project-based** atau **independent**.

Runtime	Detection	Keunggulan	Catatan
Docker	Cek Docker daemon via socket /var/run/docker.sock atau docker info	Ekosistem terbesar, Docker Compose untuk multi-container app, Docker Hub registry.	Memerlukan daemon (dockerd) berjalan sebagai root (atau rootless mode).
Podman	Cek podman binary, podman info	Rootless by default (lebih aman), daemonless, Pod sebagai unit grouping (mirip Kubernetes pod), OCI-compliant.	Kompatibel dengan Docker CLI dan Dockerfile. Podman Compose atau Quadlet untuk orchestration.

Container sebagai Resource Project

Dalam konteks project, container sering digunakan untuk menjalankan service pendukung yang tidak tersedia sebagai native service di server. Contoh umum:

Use Case	Contoh Container	Keterangan
Service pendukung project	redis:7-alpine, memcached:latest, elasticsearch:8	Service yang dibutuhkan project tapi tidak di-manage secara native oleh PRISM (karena bukan service utama server)
Aplikasi project	myapp/api:v2.1,	Aplikasi project yang di-deploy sebagai

Use Case	Contoh Container	Keterangan
(containerized)	myapp/worker:v2.1	container, biasanya via Compose
Development/staging tools	adminer, pgadmin4, mailhog, minio/console	Tools tambahan untuk development atau staging yang berjalan sebagai container
AI/ML inference	ollama, vllm, tritonserver	Model serving yang lebih mudah di-deploy sebagai container
Data pipeline	apache/airflow, prefecthq/prefect	Orchestrator pipeline yang berjalan sebagai container

5.4 Tampilan di Frontend

Process supervisors dan container runtimes ditampilkan di **Server Card** masing-masing dalam kategorinya. Setiap entry menunjukkan jumlah proses/container yang dikelola.

Contoh tampilan di Server Card:

Entry di Service List	Kategori	Status	Badge
PM2 (v5.3.0)	Process Supervisor (geekblue)	running	8 processes
Supervisor (v4.2.5)	Process Supervisor (geekblue)	running	3 programs
systemd user: deploy	Process Supervisor (geekblue)	active	5 units
Docker (v27.1.0)	Container Runtime (magenta)	running	12 containers (10 running, 2 stopped)
Podman (v5.0.2)	Container Runtime (magenta)	active	6 containers in 2 pods

Ketika entry diklik, muncul modal konfigurasi yang menampilkan daftar proses/container/unit beserta aksi yang dapat dilakukan. Detail field per modal tersedia di Lampiran A.

6. Arsitektur Keamanan

6.1 Lapisan Keamanan

Lapisan	Mekanisme	Fungsi
Transport	TLS 1.3 + Certificate Pinning	Enkripsi seluruh komunikasi dan pencegahan MITM
Identity	Mutual TLS (mTLS) + X.509 Certificate	Verifikasi identitas dua arah Hub ↔ Agent
Session	JWT (RS256) + Token Rotation	Otorisasi per request, session management
Enrollment	Pre-Shared Key + HMAC-SHA256	One-time agent onboarding
Cache Security	Valkey AUTH + TLS (opsional) + bind localhost	Proteksi akses cache layer

6.2 Hierarki Credential

Credential	Masa Berlaku	Penyimpanan
Enrollment Key (PSK)	1 jam (one-time)	Hub: SQLite (hashed). Agent: config file (sementara, dihapus setelah enrollment).
X.509 Client Certificate	1 tahun	Agent: filesystem (0600 permission). Hub: CA store.
JWT Access Token	15 menit	Agent: memory only. Frontend: httpOnly cookie.
JWT Refresh Token	24 jam (rotated)	Agent: encrypted file. Hub: Valkey + SQLite (hashed).
Hub CA Certificate	5 tahun	Hub: secure storage (HSM opsional).
Valkey AUTH password	Persistent (rotate manual)	Hub: environment variable atau secret manager.

7. Protokol Komunikasi

7.1 WebSocket Message Types

Type	Pengirim	Deskripsi
agent.hello	Agent → Hub	Registrasi awal: PSK + CSR + metadata
agent.heartbeat	Agent → Hub	Sinyal periodik (30 detik). Payload: uptime, load average.
agent.telemetry	Agent → Hub	Data service status, metrik, resource usage
agent.discovery	Agent → Hub	Hasil service discovery (initial dan periodic rescan)
agent.command_result	Agent → Hub	Konfirmasi eksekusi command + output/error
hub.command	Hub → Agent	Perintah operasional (start/stop/restart, config update)
hub.config_update	Hub → Agent	Pembaruan konfigurasi agent (scan interval, plugin enable/disable)
hub.token_refresh	Hub → Agent	JWT token baru dari refresh request

7.2 Resilience

Mekanisme	Detail Implementasi (Go)
Auto-Reconnect	Exponential backoff: 1s, 2s, 4s, ... max 5 menit. Implementasi dengan time.Sleep dan jitter random untuk mencegah thundering herd.
Offline Buffer	Agent menyimpan telemetry ke SQLite lokal (embedded) saat koneksi terputus. Background goroutine melakukan flush saat koneksi pulih.
Idempotent Commands	Setiap command memiliki UUID. Agent menyimpan set of executed command IDs (in-memory map dengan TTL). Duplikat di-skip.
Health Check	Hub mengharapkan heartbeat tiap 30 detik. Setelah 90 detik tanpa heartbeat, agent ditandai “unreachable” di SQLite + Valkey, alert dipublish.
Graceful Shutdown	Agent menangkap SIGTERM/SIGINT, mengirim agent.goodbye ke Hub, flush buffer, close WebSocket cleanly.

8. Deployment Topology

PRISM dirancang agar fleksibel dalam berbagai topologi jaringan. Bagian ini menjelaskan skenario deployment yang didukung, termasuk jaringan LAN di belakang NAT, multi-site, dan hybrid cloud.

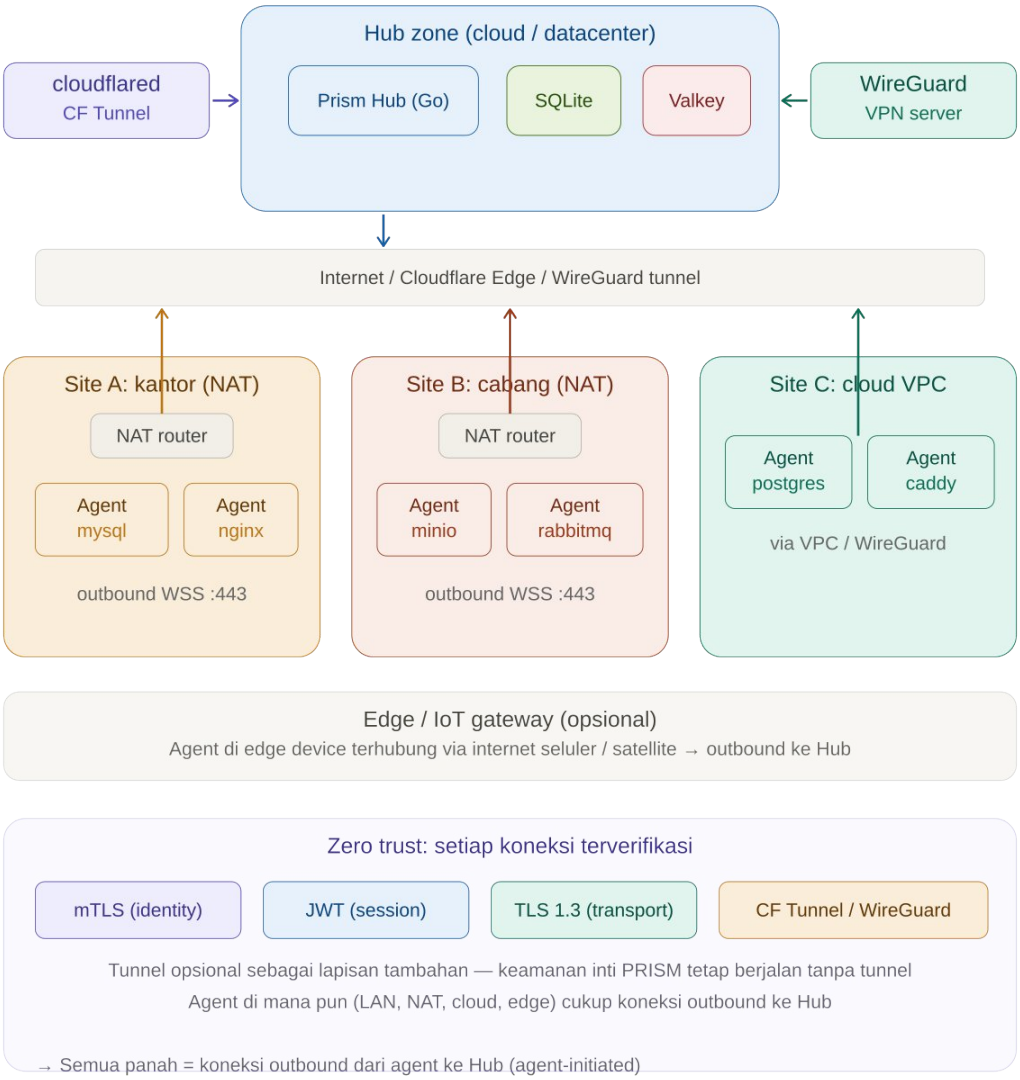
8.1 Prinsip: Agent-Initiated Outbound Connection

Seluruh arsitektur konektivitas PRISM bertumpu pada satu prinsip kunci: **agent selalu yang menginisiasi koneksi keluar (outbound) ke Hub**. Hub tidak pernah menginisiasi koneksi masuk ke agent. Prinsip ini membuat PRISM secara inheren kompatibel dengan NAT, firewall, dan jaringan yang restriktif.

Begitu koneksi WebSocket terbentuk, channel tersebut bersifat **persistent dan bidirectional** — Hub dapat mengirim command ke agent melalui koneksi yang sama tanpa perlu membuka port inbound di sisi agent. Heartbeat setiap 30 detik berfungsi ganda: sebagai health check sekaligus menjaga agar NAT mapping tidak expired (kebanyakan NAT timeout 60–300 detik).

Gambar 4: Deployment Topology — NAT, Tunnel, dan Multi-Site

PRISM deployment topology: NAT, tunnel, dan multi-site



8.2 Skenario Deployment

Skenario 1: Single LAN (Sederhana)

Hub dan seluruh agent berada dalam satu jaringan LAN yang sama. Ini adalah skenario paling sederhana, cocok untuk small office atau lab environment.

Aspek	Detail
Topologi	Hub di LAN (misalnya 192.168.1.10). Agent di server lain dalam LAN yang sama.
Hub URL di Agent	wss://192.168.1.10:443 atau wss://prism.local:443
NAT	Tidak ada NAT antara agent dan Hub.
Firewall	Hanya perlu allow outbound port 443 dari agent ke Hub (biasanya sudah terbuka).
Persyaratan	Hub harus memiliki IP statis atau hostname yang resolvable di LAN.

Skenario 2: Agent di LAN Belakang NAT

Skenario umum di mana agent berada di jaringan LAN kantor/site yang menggunakan NAT untuk akses internet, sementara Hub berada di cloud atau datacenter dengan public IP.

Aspek	Detail
Topologi	Hub di cloud/datacenter (public IP atau domain). Agent di LAN belakang NAT router.
Hub URL di Agent	wss://prism.smartsystem.id:443 (domain publik)
NAT Traversal	Tidak diperlukan konfigurasi khusus. Agent membuka koneksi outbound yang melewati NAT secara natural, sama seperti browser membuka website.
Firewall	Allow outbound HTTPS (port 443) dari LAN ke internet. Biasanya sudah default-allow di kebanyakan jaringan.
Port Forwarding	TIDAK diperlukan. Agent tidak membutuhkan port inbound.
Multiple Agents per LAN	Didukung penuh. Setiap agent membuka koneksi outbound sendiri. NAT router melacak masing-masing connection.
Aggressive NAT	Jika NAT timeout < 30 detik (misalnya carrier-grade NAT), interval heartbeat perlu diperkecil ke 15 atau 20 detik.

Skenario 3: Multi-Site / Branch Office

Organisasi memiliki beberapa lokasi (kantor pusat, cabang, gudang, pabrik) dengan masing-masing memiliki LAN sendiri dan server-server lokal.

Aspek	Detail
Topologi	Hub terpusat di cloud/datacenter. Agent tersebar di berbagai site, masing-masing di belakang NAT/firewall site tersebut.
Konektivitas	Setiap site hanya memerlukan koneksi internet outbound ke Hub. Tidak perlu VPN site-to-site khusus untuk PRISM.
Identifikasi Site	Agent metadata menyertakan site/location identifier. Hub dapat mengelompokkan server berdasarkan site di dashboard.
Bandwidth	Minimal. Heartbeat 30 detik + telemetry 60 detik = estimasi < 5 KB/menit per agent. Ratusan agent di satu site < 500 KB/menit.
Offline Resilience	Jika link internet site terputus, agent menyimpan data ke offline buffer. Otomatis flush saat koneksi pulih.

Skenario 4: Hybrid Cloud

Organisasi menjalankan server di kombinasi on-premise datacenter, public cloud (AWS, GCP, Azure), dan edge location.

Aspek	Detail
Topologi	Hub di salah satu cloud atau dedicated server. Agent di on-premise (belakang NAT), di VM cloud (VPC), dan di edge device.
Cloud VM Agent	Agent di cloud VM terhubung ke Hub melalui internet atau private network (VPC peering/private link jika Hub di cloud yang sama).
On-premise Agent	Terhubung via internet outbound, sama seperti skenario NAT.
Edge Agent	Terhubung via internet (seluler/satellite/fixed). Heartbeat interval mungkin perlu disesuaikan untuk koneksi yang tidak stabil.
Security	Seluruh koneksi menggunakan mTLS + JWT terlepas dari jenis jaringan. Zero trust: tidak ada perbedaan trust level antara cloud dan on-premise agent.

9. Zero Trust Network dan Secure Tunnel

Meskipun arsitektur outbound-only PRISM sudah cukup untuk sebagian besar skenario, organisasi dengan kebijakan keamanan ketat mungkin memerlukan lapisan tambahan. Bagian ini menjelaskan integrasi PRISM dengan pendekatan **Zero Trust Network Access (ZTNA)** dan secure tunnel.

9.1 Prinsip Zero Trust di PRISM

PRISM sudah menerapkan beberapa prinsip Zero Trust secara built-in:

Prinsip Zero Trust	Implementasi di PRISM
Never trust, always verify	Setiap request (agent maupun admin) harus membawa bukti identitas: mTLS certificate + JWT token. Tidak ada implicit trust berdasarkan posisi jaringan.
Least privilege access	JWT token berisi scope spesifik per agent. Agent hanya bisa mengakses resource yang relevan dengan server-nya. RBAC untuk admin di Web App.
Assume breach	Certificate dapat di-revoke secara instan. JWT berumur pendek (15 menit). Compromised agent tidak mempengaruhi agent lain. Audit log immutable.
Verify explicitly	Setiap command dari Hub ke agent divalidasi ulang: signature check, agent ID match, command authorization. Agent memverifikasi bahwa command berasal dari Hub yang sah via mTLS.
Micro-segmentation	Setiap agent adalah unit terisolasi. Agent A tidak bisa berkomunikasi dengan Agent B. Semua komunikasi harus melalui Hub yang menegakkan policy.

9.2 Kapan Memerlukan Secure Tunnel?

Meskipun koneksi outbound PRISM sudah aman (mTLS + JWT over TLS 1.3), ada skenario di mana organisasi memerlukan secure tunnel tambahan:

Skenario	Masalah	Solusi Tunnel
Firewall sangat restriktif	Outbound WebSocket (bahkan port 443) diblokir oleh corporate firewall atau deep packet inspection (DPI) yang memblokir WebSocket upgrade.	Cloudflare Tunnel (cloudflared) atau WireGuard melewati restriksi.
Compliance	Regulasi mengharuskan	WireGuard sebagai transport layer tambahan.

Skenario	Masalah	Solusi Tunnel
requirement	seluruh traffic melewati VPN atau encrypted tunnel tertentu, meskipun PRISM sudah menggunakan mTLS.	
Hub tidak memiliki public IP	Hub berada di jaringan private/on-premise tanpa public IP, tetapi agent tersebar di luar jaringan.	Cloudflare Tunnel meng-expose Hub tanpa public IP.
Defense in depth	Organisasi menginginkan enkripsi berlapis: TLS 1.3 (PRISM) di dalam WireGuard tunnel.	WireGuard + PRISM mTLS = double encryption.
Network segmentation	Agent di segmen jaringan yang berbeda, dipisahkan oleh firewall internal yang ketat.	WireGuard mesh atau Cloudflare WARP menghubungkan segmen.

9.3 Opsi 1: Cloudflare Tunnel (cloudflared)

Cloudflare Tunnel memungkinkan Hub di-expose ke internet tanpa membuka port inbound dan tanpa public IP. Tunnel daemon (cloudflared) berjalan di sisi Hub dan membuat koneksi outbound ke edge network Cloudflare.

Arsitektur dengan Cloudflare Tunnel

Komponen	Lokasi	Fungsi
cloudflared daemon	Co-located dengan Hub	Membuat persistent tunnel dari Hub ke Cloudflare edge. Hub API dan WebSocket di-route melalui tunnel.
Cloudflare Edge	Cloudflare network (global)	Menerima koneksi dari agent, melakukan TLS termination (opsional), forward ke Hub via tunnel.
Agent	Di mana saja (LAN, cloud, edge)	Terhubung ke prism.smartsystem.id yang di-resolve ke Cloudflare edge. Agent tidak perlu tahu bahwa Hub ada di belakang tunnel.

Keuntungan Cloudflare Tunnel

Aspek	Detail
Tanpa Public IP	Hub tidak memerlukan public IP. cloudflared membuat koneksi outbound ke Cloudflare, jadi Hub juga bisa di belakang NAT.
Zero-config DNS	Cloudflare otomatis mengatur DNS record untuk domain tunnel. prism.smartsystem.id langsung mengarah ke tunnel.
DDoS Protection	Traffic melewati Cloudflare edge yang menyediakan DDoS mitigation, WAF, dan rate limiting secara gratis.
Cloudflare Access (ZTNA)	Dapat diintegrasikan dengan Cloudflare Access untuk menambahkan identity-aware proxy di depan Hub. Admin Web App harus autentikasi via SSO/IdP sebelum mencapai Hub.
WebSocket Support	Cloudflare Tunnel mendukung WebSocket passthrough secara penuh. Koneksi persistent agent tetap berfungsi.
Audit & Visibility	Cloudflare dashboard menyediakan log koneksi, analytics traffic, dan security event.

Konfigurasi Cloudflare Tunnel untuk PRISM

Step	Aksi	Detail
1	Install cloudflared di server Hub	Download binary atau install via package manager. cloudflared login untuk autentikasi ke akun Cloudflare.
2	Buat tunnel	cloudflared tunnel create prism-hub. Menghasilkan tunnel ID dan credentials file.

Step	Aksi	Detail
3	Konfigurasi routing	config.yml: route traffic dari prism.smartsystem.id ke localhost:443 (Hub API + WebSocket). Pastikan opsi noTLSVerify: false agar mTLS tetap berjalan end-to-end.
4	Jalankan sebagai service	cloudflared service install. Tunnel berjalan sebagai systemd service, auto-start saat boot.
5	Konfigurasi agent	Agent cukup menggunakan hub_url: wss://prism.smartsystem.id:443. Tidak perlu konfigurasi tunnel di sisi agent.
6	Opsional: Cloudflare Access	Tambahkan Access Application untuk Web App. Admin harus autentikasi via SSO (Google, Okta, Azure AD) sebelum mengakses dashboard.

mTLS End-to-End melalui Cloudflare Tunnel

Cloudflare Tunnel mendukung passthrough mode di mana TLS tidak di-terminate di edge. Ini memastikan mTLS antara agent dan Hub tetap end-to-end. Cloudflare edge hanya meneruskan encrypted traffic tanpa melihat payload. Untuk konfigurasi ini, gunakan opsi originServerName dan noTLSVerify: false di config tunnel.

9.4 Opsi 2: WireGuard VPN

WireGuard adalah modern VPN protocol yang ringan, cepat, dan mudah dikonfigurasi. WireGuard membentuk encrypted tunnel di layer 3 (network layer) yang menghubungkan agent dan Hub dalam satu virtual private network.

Arsitektur dengan WireGuard

Komponen	Lokasi	Fungsi
WireGuard Server	Co-located dengan Hub (atau dedicated gateway)	Menerima koneksi WireGuard dari agent. Assign IP address dalam subnet virtual (misalnya 10.100.0.0/16).
WireGuard Client	Di setiap server yang menjalankan agent	Membuat tunnel ke WireGuard server. Agent berkomunikasi dengan Hub melalui IP virtual di dalam tunnel.
Hub	Mendengarkan di IP virtual WireGuard (misalnya 10.100.0.1)	Menerima koneksi WebSocket dari agent melalui tunnel. Tidak perlu exposed ke public internet.

Model Deployment WireGuard

Model	Topologi	Use Case
Star (Hub-and-Spoke)	WireGuard server di Hub. Semua agent terhubung ke satu server.	Paling sederhana. Cocok untuk < 100 agent. Hub menjadi satu-satunya entry point.
Mesh (via WireGuard tools)	Setiap node saling terhubung. Menggunakan tools seperti wg-mesh, netbird, atau tailscale (berbasis WireGuard).	Untuk skenario di mana agent perlu berkomunikasi peer-to-peer (bukan kebutuhan PRISM, tapi berguna jika ada service lain).
Gateway per Site	Satu WireGuard gateway per site/branch. Agent di site terhubung ke gateway lokal, gateway terhubung ke Hub.	Untuk multi-site dengan banyak agent per site. Mengurangi jumlah tunnel ke Hub.

Keuntungan WireGuard

Aspek	Detail
Performa	Kernel-space implementation, overhead minimal (< 5% throughput loss). Latency sangat rendah dibanding OpenVPN atau IPSec.
Kesederhanaan	Konfigurasi minimalis: sepasang public/private key, endpoint, allowed IPs. Tidak ada complex PKI atau certificate management.
Roaming	Agent yang berpindah IP (misalnya laptop, edge device dengan koneksi seluler) tetap terhubung. WireGuard otomatis re-establish tunnel.

Aspek	Detail
Stealth	WireGuard tidak merespons paket dari IP yang tidak dikenal. Port terlihat “closed” dari luar, mengurangi attack surface.
Cross-platform	Tersedia untuk Linux (kernel module), Windows, macOS, Android, iOS, FreeBSD.
Double Encryption	Traffic PRISM (mTLS + TLS 1.3) berjalan di dalam WireGuard tunnel. Dua lapisan enkripsi independen.

Konfigurasi WireGuard untuk PRISM

Step	Server (Hub side)	Client (Agent side)
1. Generate keys	wg genkey | tee server_private.key | wg pubkey > server_public.key	wg genkey | tee agent_private.key | wg pubkey > agent_public.key
2. Interface config	[Interface] Address = 10.100.0.1/16, ListenPort = 51820, PrivateKey = <server_private>	[Interface] Address = 10.100.0.x/32, PrivateKey = <agent_private>
3. Peer config	[Peer] PublicKey = <agent_public>, AllowedIPs = 10.100.0.x/32	[Peer] PublicKey = <server_public>, Endpoint = hub-public-ip:51820, AllowedIPs = 10.100.0.1/32, PersistentKeepalive = 25
4. Start	wg-quick up wg0 (atau systemd: systemctl enable wg-quick@wg0)	wg-quick up wg0
5. Agent config	—	hub_url: wss://10.100.0.1:443 (IP virtual Hub di dalam tunnel)

PersistentKeepalive untuk NAT Traversal

Parameter PersistentKeepalive = 25 pada client config mengirim keepalive packet setiap 25 detik, menjaga NAT mapping tetap aktif. Ini penting jika agent berada di belakang NAT. Tanpa keepalive, NAT mapping bisa expired dan Hub tidak bisa mengirim data ke agent melalui tunnel.

9.5 Perbandingan: Cloudflare Tunnel vs WireGuard

Aspek	Cloudflare Tunnel	WireGuard
Setup complexity	Rendah. Install cloudflared, buat tunnel, selesai. Agent tidak perlu konfigurasi tambahan.	Sedang. Perlu generate key pair per agent, distribusi public key, konfigurasi peer.
Konfigurasi agent	Tidak ada perubahan. Agent tetap konek ke domain publik.	Agent harus install WireGuard client dan dikonfigurasi tunnel.
Public IP requirement	Hub tidak perlu public IP. cloudflared membuat outbound tunnel.	WireGuard server perlu public IP atau accessible endpoint (kecuali menggunakan relay).
Overhead / Latency	Sedikit lebih tinggi (traffic via Cloudflare edge, tambahan hop).	Minimal (kernel-space, direct tunnel antara agent dan Hub).
Biaya	Gratis untuk Cloudflare Tunnel. Cloudflare Access (ZTNA) gratis hingga 50 user.	Gratis dan open-source. Tidak ada biaya lisensi.
Ekstra fitur	DDoS protection, WAF, Cloudflare Access (SSO/IdP integration), analytics, bot protection.	Pure encrypted tunnel. Tidak ada fitur tambahan (security ditangani oleh PRISM sendiri).
Offline / air-gapped	Tidak bisa. Memerlukan koneksi ke Cloudflare network.	Bisa. WireGuard berjalan peer-to-peer tanpa ketergantungan ke layanan pihak ketiga.
Scalability	Cloudflare menangani routing dan load balancing. Sangat scalable.	Perlu manajemen peer config per agent. Tools seperti Netbird/Headscale dapat membantu.
Vendor dependency	Bergantung pada Cloudflare (SaaS).	Tidak ada vendor dependency. Fully self-hosted.

9.6 Rekomendasi Pemilihan

Skenario	Rekomendasi	Alasan
Hub di cloud, agent tersebar, mau cepat setup	Cloudflare Tunnel	Zero config di sisi agent. Hub langsung accessible via domain. Bonus DDoS protection dan Cloudflare Access.
Hub tanpa public IP, perlu ZTNA	Cloudflare Tunnel + Access	Tunnel menghilangkan kebutuhan public IP. Access menambahkan SSO layer untuk admin Web App.
Environment air-gapped atau regulated	WireGuard	Tidak ada dependency ke layanan pihak ketiga. Seluruh infrastruktur dikelola sendiri.
Perlu performa tunnel maksimal	WireGuard	Kernel-space, minimal overhead, latency rendah.
Multi-site dengan banyak agent per site	WireGuard (gateway per site)	Satu WireGuard gateway per site, mengurangi jumlah tunnel ke Hub.

Skenario	Rekomendasi	Alasan
Tidak ada kebutuhan khusus	Tanpa tunnel (default PRISM)	Arsitektur outbound-only PRISM sudah NAT-friendly dan aman (mTLS + JWT). Tunnel hanya diperlukan untuk skenario spesifik.

Catatan Penting

Tunnel (Cloudflare atau WireGuard) adalah lapisan transport tambahan. Keamanan inti PRISM (mTLS + JWT + enrollment) tetap berjalan di atasnya. Ini berarti jika tunnel ter-compromise, komunikasi PRISM tetap terenkripsi dan terotentikasi. Defense in depth: dua lapisan enkripsi independen.

Lampiran A: Detail Modal Konfigurasi per Service

Lampiran ini mendokumentasikan field dan aksi spesifik yang ditampilkan di modal konfigurasi untuk setiap service yang didukung PRISM.

A.1 MySQL

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port, Socket, Data Dir, Server ID, Version, Uptime	Read-only (Descriptions)	Info dasar instance
Overview	Buffer Pool Size, Threads, Connections, Max Connections	Read-only (Statistic)	Metrik performa real-time
Accounts	User list: username, host, auth plugin, default schema	Table + CRUD (Drawer)	Manajemen user MySQL
Accounts	Privileges: global, database, table level	Transfer/Checkbox dalam Drawer	Grant/Revoke per user
Accounts	Kategorisasi: project / independent	Tag + Select	Assign user ke project
Config	Database list: nama, size, charset, collation, tables	Table + Create/Drop	Inventory database
Config	Replication: role, binlog pos, slave lag	Descriptions (Read-only)	Status replikasi
Config	Backup: last backup, method, size	Descriptions + Button (trigger)	Info + trigger manual backup

A.2 PostgreSQL

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port, Data Dir, Cluster Name, Version, Uptime	Read-only	Info dasar
Overview	Shared Buffers, Work Mem, Active Conns, Max Conns, WAL Level	Statistic	Metrik performa
Accounts	Role list: name, superuser, login, conn limit	Table + CRUD	Manajemen roles
Accounts	Privileges: DB ownership, schema usage, table	Edit (Grant/Revoke)	Permission per role

Tab	Field / Informasi	Tipe	Keterangan
	perms		
Config	Database list: name, owner, size, encoding, tablespace	Table + Create/Drop	Inventory DB
Config	Schema list per DB: name, owner, table count	Table + Create/Drop	Inventory schema
Config	Extensions: installed, available	Table + Install/Uninstall	Extension management
Config	Replication: streaming status, slots, subscriber lag	Read-only	Monitoring replikasi

A.3 MongoDB

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port, Data Path, Version, Uptime, Storage Engine	Read-only	Info dasar
Overview	Connections, Memory (resident/virtual)	Statistic	Metrik
Accounts	User list: username, DB scope, roles	Table + CRUD	User management
Accounts	Roles: built-in, custom, privilege actions	Edit	Role management
Config	Database list: name, size, collections, indexes	Table + Create/Drop	Inventory DB
Config	Collection list per DB: name, docs, avg size, indexes	Read-only Table	Detail collection
Config	Replica Set: members, primary/secondary, oplog, lag	Read-only	Status RS

A.4 Valkey

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port, Version, Uptime, Mode (standalone/cluster/sentinel)	Read-only	Info dasar
Overview	Memory Used/Peak, Clients, Keys, Hit Rate	Statistic + Progress	Metrik

Tab	Field / Informasi	Tipe	Keterangan
Accounts	ACL User list: name, enabled, commands, key patterns	Table + CRUD	ACL management
Config	Persistence: RDB last save, AOF status	Read-only + Button (trigger BGSAVE)	Persistence status
Config	Replication: role, connected replicas, offset	Read-only	Status replikasi
Config	Keyspace: DB index, keys, expires, avg TTL	Read-only Table	Key distribution

A.5 RabbitMQ

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port (AMQP/Management), Version, Uptime, Erlang Version	Read-only	Info dasar
Overview	Queues, Messages, Rate (pub/deliver), Connections, Channels	Statistic	Metrik
Accounts	User list: name, tags, password hash algo	Table + CRUD	User management
Accounts	VHost permissions: configure/write/read regex	Edit per user per vhost	Permission management
Config	Virtual Host list: name, messages, queues, connections	Table + CRUD	VHost management
Config	Queue list per VHost: name, type, messages, consumers, state	Table + Create/Delete/Purge	Queue management
Config	Exchange list: name, type, bindings	Table + Create/Delete	Exchange management
Config	Policies: name, pattern, apply-to, definition	Table + CRUD	Policy management

A.6 MQTT (Mosquitto)

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port (MQTT/WS), Version, Uptime	Read-only	Info dasar

Tab	Field / Informasi	Tipe	Keterangan
Overview	Clients, Subscriptions, Messages In/Out, Retained	Statistic	Metrik
Accounts	User list: username, enabled/disabled	Table + CRUD	User management
Accounts	Topic ACL: pattern, access type, priority	Edit per user	Topic access control
Config	Listeners: port, protocol, max connections	Read-only	Listener info
Config	Bridge: remote broker, topics, QoS, status	Read-only	Bridge status

A.7 MinIO

Tab	Field / Informasi	Tipe	Keterangan
Overview	API Port, Console Port, Version, Mode, Uptime	Read-only	Info dasar
Overview	Total/Used/Available Storage, Buckets, Objects	Statistic + Progress	Kapasitas
Accounts	Access Key list: ID, status, created, last used	Table + CRUD	Key management
Accounts	Policy per key: built-in atau custom JSON	Edit (code editor)	IAM policy
Accounts	Service Accounts: parent, policy, session duration	Table + CRUD	Service account mgmt
Config	Bucket list: name, size, objects, versioning, lock	Table + CRUD	Bucket management
Config	Bucket Policy: public/private/custom	Edit per bucket	Access policy
Config	Lifecycle Rules: expiration, transition, filter	Table + CRUD per bucket	Lifecycle management
Config	Event Notification: target, events, filter	Table + CRUD	Notification config

A.8 Garage

Tab	Field / Informasi	Tipe	Keterangan
Overview	API Port, Admin Port, Version, Cluster Status	Read-only	Info dasar

Tab	Field / Informasi	Tipe	Keterangan
Overview	Node list: ID, hostname, zone, capacity, status	Table (Read-only)	Cluster nodes
Accounts	Key list: ID, name, access key, status	Table + CRUD	API key management
Accounts	Bucket permission per key: read/write/owner	Edit	Permission per bucket
Config	Bucket list: name, size, objects, quotas	Table + CRUD	Bucket management
Config	Cluster Layout: replication factor, zones	Read-only	Layout info

A.9 SeaweedFS

Tab	Field / Informasi	Tipe	Keterangan
Overview	Master/Filer/S3 Port, Version, Topology	Read-only	Info dasar
Overview	Volumes, Free Volumes, Total Size, Volume Servers	Statistic	Kapasitas
Accounts	S3 Access Key: ID, secret (masked), actions	Table + CRUD	S3 key management
Accounts	IAM Policy per key: bucket permissions	Edit	Policy management
Config	S3 Bucket list: name, size, objects	Table + CRUD	Bucket management
Config	Filer Config: collections, replication, TTL	Read-only	Filer info
Config	EC Status: encoded volumes, shards	Read-only	Erasure coding status

A.10 vsftpd

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port (FTP/FTPS), Version, Uptime, Anonymous (on/off)	Read-only	Info dasar
Overview	Active Connections, Sessions Today	Statistic	Metrik
Accounts	User list: name, home dir, status, bandwidth limit	Table + CRUD	FTP user management
Accounts	Permissions per user: read/write/mkdir/delete	Edit	Hak akses
Config	TLS: cert path, min version, force encryption	Read-only	Enkripsi
Config	Passive Mode: port range, external IP	Read-only	Passive config
Config	Chroot: jail status, allowed dirs	Read-only	Isolasi directory

A.11 SFTPGo

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port (SFTP/HTTP/FTP/WebDAV), Version, Uptime	Read-only	Info dasar
Overview	Active Connections, Transfers Today, Data Transferred	Statistic	Metrik
Accounts	User list: name, status, home dir, quota, bandwidth, last login	Table + CRUD	User management
Accounts	Permissions per directory: list/download/upload/delete/rename/etc	Edit (Tree + Checkbox)	Granular permissions
Accounts	Public Keys per user: fingerprint, added date	Table + CRUD	SSH key management
Accounts	Virtual Folders: name, mapped path, quota	Table + CRUD	Folder mapping
Config	Data Provider: backend type, status	Read-only	Backend info
Config	Event Rules: trigger, conditions, actions	Table + CRUD	Event automation
Config	IP Filter: allow/deny list	Table + CRUD	IP access control

A.12 Nginx

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port (HTTP/HTTPS), Version, Uptime, Workers, Config Path	Read-only	Info dasar
Overview	Active Conns, Req/sec, Reading/Writing/Waiting	Statistic	Metrik (stub_status)
Config	Site list: domain, port, root/proxy_pass, SSL, enabled	Table + Enable/Disable	Virtual host management
Config	Upstream per site: backends, LB method, health check	Read-only	Reverse proxy info
Config	SSL per site: issuer, expiry, auto-renew	Read-only + Tag (warning jika < 30d)	Certificate monitoring
Config	Config Test: nginx -t output	Button + Result display	Validasi konfigurasi

A.13 Caddy

Tab	Field / Informasi	Tipe	Keterangan
Overview	Port (HTTP/HTTPS/Admin), Version, Uptime	Read-only	Info dasar
Overview	Active Requests, Total Requests	Statistic	Metrik
Config	Site list: domain, upstream/root, TLS mode, status	Table + Edit	Site management
Config	Reverse Proxy: upstream, health check, LB, headers	Edit per site	Proxy config
Config	TLS/Auto HTTPS: ACME, certificates, renewal status	Read-only	HTTPS status
Config	Middleware: encode, rate limit, basic auth, IP filter	Edit per site	Middleware config

A.14 UFW

Tab	Field / Informasi	Tipe	Keterangan
Overview	Status (active/inactive), Default Policy, Logging Level	Read-only + Tag	Firewall status

Tab	Field / Informasi	Tipe	Keterangan
Config	Rule list: action, direction, protocol, port, from/to, comment	Table + CRUD	Rule management
Config	App Profiles: name, title, ports	Read-only Table	Application profiles
Config	Enable/Disable	Switch (Popconfirm)	Toggle firewall

A.15 firewalld

Tab	Field / Informasi	Tipe	Keterangan
Overview	Status, Default Zone, Log Denied, Panic Mode	Read-only	Firewall status
Config	Zone list: name, target, interfaces, sources	Table + Edit	Zone management
Config	Services per zone	Add/Remove (Transfer)	Service assignment
Config	Port rules per zone: port, protocol, permanent/runtime	Table + CRUD	Port management
Config	Rich Rules: family, source, dest, action, log, limit	Table + CRUD	Rich rule management
Config	Reload (Runtime/Permanent)	Button	Apply changes

A.16 iptables

Tab	Field / Informasi	Tipe	Keterangan
Overview	Active tables, Total rules per table, Kernel module	Read-only	Status dasar
Config	Chain list per table: name, policy, packet/byte counters	Table + Create/Delete (custom)	Chain management
Config	Rules per chain: #, target, proto, src, dst, match, comment	Table + Insert/Append/Delete	Rule management
Config	NAT rules: SNAT, DNAT, MASQUERADE, REDIRECT	Table + CRUD	NAT config

Tab	Field / Informasi	Tipe	Keterangan
Config	Save/Restore: export, import	Button (download/upload)	Persist rules

A.17 nftables

Tab	Field / Informasi	Tipe	Keterangan
Overview	Active tables, chains, rules count, ruleset size	Read-only	Status dasar
Config	Table list: name, family (ip/ip6/inet/arp/bridge/netdev)	Table + Create/Delete	Table management
Config	Chain list per table: name, type, hook, priority, policy	Table + CRUD	Chain management
Config	Rules per chain: handle, expression, counter, comment	Table + CRUD	Rule management
Config	Sets & Maps: name, type, elements, flags	Table + CRUD	Set/map management
Config	Export/Import ruleset	Button (download/upload)	Ruleset backup

A.18 CrowdSec

Tab	Field / Informasi	Tipe	Keterangan
Overview	Version, Uptime, Local API, Machines, Console Status	Read-only	Info dasar
Overview	Alerts (day/week/month), Active Decisions, Top Attackers	Statistic	Security metrics
Config	Decisions: IP, type (ban/captcha/throttle), duration, reason	Table + Delete (unban)	Decision management
Config	Scenarios: name, status, description, event count	Table + Enable/Disable	Scenario management
Config	Bouncers: name, type, IP, last pull, status	Table + Register/Delete	Bouncer management
Config	Parsers: name, status, stage	Table + Enable/Disable	Parser management
Config	Allow List: IP/range, reason, date	Table + CRUD	Whitelist

Tab	Field / Informasi	Tipe	Keterangan
Config	Collections: name, author, contents	Table + Install/Remove	Hub collection mgmt

A.19 PM2

Tab	Field / Informasi	Tipe	Keterangan
Overview	PM2 Version, Daemon PID, Node.js Version, PM2 Home Path	Read-only	Info dasar PM2 daemon
Overview	Total Processes, Running, Stopped, Errored, Total Restarts	Statistic	Metrik agregat
Config	Process list: id, name, mode (fork/cluster), instances, status, CPU, memory, uptime, restarts, script path	Table	Daftar seluruh proses yang dikelola PM2
Config	Per process actions: Start, Stop, Restart, Reload (0-downtime), Delete	Button + Popconfirm	Aksi per proses
Config	Ecosystem file: path, contents (editable)	Read + Edit (code editor)	Edit ecosystem.config.js
Config	Environment variables per process: key-value pairs	Table (Read-only)	Env vars yang di-set untuk proses
Config	Log paths per process: stdout log, stderr log	Read-only + Link (live tail)	Path log dengan akses ke live tail
Config	Startup script: status (enabled/disabled), init system	Read-only + Button (generate)	pm2 startup / pm2 save status
Config	Flush Logs, Reset Restart Count, Save Process List	Action buttons	Maintenance actions

A.20 Supervisor (supervisord)

Tab	Field / Informasi	Tipe	Keterangan
Overview	Supervisor Version, PID, Config Path, State (RUNNING/FATAL)	Read-only	Info dasar supervisord
Overview	Total Programs, Running, Stopped, Fatal	Statistic	Metrik agregat
Config	Program list: name, state (RUNNING/STOPPED/STARTING/BACKOFF/FATAL),	Table	Daftar program yang dikelola

Tab	Field / Informasi	Tipe	Keterangan
	PID, uptime, group		
Config	Per program actions: Start, Stop, Restart, Clear Log, Tail Log	Button + Popconfirm	Aksi per program
Config	Program config per entry: command, directory, user, autostart, autorestart, startsecs, startretries, redirect_stderr, stdout_logfile, environment	Descriptions (Read-only)	Konfigurasi program dari .conf file
Config	Process groups: group name, member programs, priority	Collapse + Table	Manajemen process groups
Config	Reload Config (supervisorctl reread + update)	Action button	Menerapkan perubahan config tanpa restart supervisord

A.21 systemd User Services

Khusus untuk systemd user services: PRISM hanya mengelola unit yang berjalan di scope **user** (systemctl --user), bukan system-wide. Agent menggunakan systemctl --user --machine=<username>@ untuk mengakses user service context.

Tab	Field / Informasi	Tipe	Keterangan
Overview	User context: username, UID, lingering status (loginctl enable-linger), user unit dir path	Read-only	Info dasar user service context
Overview	Total Units, Active, Inactive, Failed	Statistic	Metrik agregat per user
Config	Unit list: unit name, load state (loaded/not-found), active state (active/inactive/failed), sub state (running/dead/exited), description	Table	Daftar systemd user units
Config	Per unit actions: Start, Stop, Restart, Enable, Disable, Mask, Unmask	Button + Popconfirm	Aksi per unit
Config	Unit file content: [Unit], [Service], [Install] sections	Descriptions atau Read-only code view	Isi dari unit file (~/.config/systemd/user/*.service)
Config	Unit properties: ExecStart, WorkingDirectory,	Descriptions (Read-only)	Properties penting dari unit

Tab	Field / Informasi	Tipe	Keterangan
	Environment, Restart policy, resource limits (MemoryMax, CPUQuota)		
Config	Journal log per unit: journalctl --user-unit=<name>	Live tail (monospace)	Log output dari unit via journald
Config	Daemon Reload (systemctl --user daemon-reload)	Action button	Reload setelah edit unit file
Config	Enable Lingering	Switch + Popconfirm	loginctl enable-linger <user> agar user services tetap berjalan tanpa login session

A.22 Docker

Tab	Field / Informasi	Tipe	Keterangan
Overview	Docker Version, API Version, Storage Driver, Cgroup Driver, Runtime, Root Dir, Server OS	Read-only	Info dasar Docker daemon
Overview	Total Containers (running/paused/stopped), Total Images, Total Volumes, Total Networks	Statistic	Metrik agregat
Overview	Disk Usage: images size, containers size, volumes size, build cache size	Statistic + Progress	Penggunaan disk oleh Docker
Config	Container list: name, image, status, ports, created, networks, mounts, command	Table (filterable, sortable)	Daftar seluruh container
Config	Per container actions: Start, Stop, Restart, Pause, Unpause, Remove (force option), View Logs	Button + Popconfirm	Aksi per container
Config	Container inspect: environment vars, labels, resource limits (CPU, memory), restart policy, health check config	Descriptions / Collapse	Detail konfigurasi container
Config	Container stats (live): CPU %, Memory usage/limit, Network I/O, Block I/O, PIDs	Table (auto-refresh) atau Statistic	Real-time resource monitoring per container

Tab	Field / Informasi	Tipe	Keterangan
Config	Image list: repository, tag, image ID, size, created	Table + Remove (prune option)	Manajemen images
Config	Volume list: name, driver, mountpoint, size, used by containers	Table + Create/Remove	Manajemen volumes
Config	Network list: name, driver (bridge/host/overlay), subnet, gateway, connected containers	Table + Create/Remove	Manajemen networks
Config	Docker Compose projects (jika terdeteksi): project name, services, status, compose file path	Collapse + Table	Daftar compose project dengan status per service
Config	Compose actions per project: Up, Down, Restart, Pull, View Config	Button + Popconfirm	Aksi level compose project
Config	System Prune: remove unused containers, images, networks, volumes	Button (Popconfirm, destructive)	Cleanup disk space
Logs	Container log stream: per container, follow mode, tail lines, since/until filter	Live tail (monospace) + Select (container picker)	Log viewer per container

Docker Compose Integration

Jika agent mendeteksi adanya docker-compose.yml atau compose.yaml di lokasi tertentu, PRISM secara otomatis mengelompokkan container yang terkait sebagai satu Compose project. Admin dapat melakukan operasi level project (up/down/restart) selain operasi level individual container. Compose project juga dapat di-tag sebagai project-based atau independent.

A.23 Podman

Podman memiliki API yang kompatibel dengan Docker, sehingga modal konfigurasinya sangat mirip. Perbedaan utama: Podman bersifat **rootless by default** (tidak memerlukan daemon dengan root privilege) dan mendukung **pod** sebagai unit pengelompokan container.

Tab	Field / Informasi	Tipe	Keterangan
Overview	Podman Version, API Version, Runtime (crun/runc), Storage Driver, Graph Root, Run Root	Read-only	Info dasar Podman

Tab	Field / Informasi	Tipe	Keterangan
Overview	Total Containers (running/paused/stopped), Total Images, Total Volumes, Total Pods	Statistic	Metrik agregat
Overview	Rootless mode: active/inactive, user context	Tag + Descriptions	Informasi privilege mode
Config	Container list: name, image, status, ports, pod (jika ada), created, networks, mounts	Table (filterable, sortable)	Daftar container (mirip Docker)
Config	Per container actions: Start, Stop, Restart, Pause, Remove, View Logs	Button + Popconfirm	Aksi per container
Config	Container inspect: env vars, labels, resource limits, restart policy, health check, security options (SELinux, seccomp)	Descriptions / Collapse	Detail container (termasuk security options Podman-specific)
Config	Container stats (live): CPU %, Memory, Network I/O, Block I/O	Table (auto-refresh)	Real-time monitoring
Config	Pod list: name, status, infra container ID, container count, created	Table + Create/Remove	Manajemen pods
Config	Per pod actions: Start, Stop, Restart, Remove, View Containers in Pod	Button + Popconfirm	Aksi level pod
Config	Image list: repository, tag, ID, size, created	Table + Pull/Remove	Manajemen images
Config	Volume list: name, driver, mountpoint, size	Table + Create/Remove	Manajemen volumes
Config	Network list: name, driver (bridge/macvlan), subnet, connected containers	Table + Create/Remove	Manajemen networks
Config	Podman Compose / Quadlet: compose file atau systemd-based container management	Collapse + Table	Jika menggunakan podman-compose atau Quadlet (.container files)
Config	System Prune	Button (Popconfirm)	Cleanup unused resources
Logs	Container/Pod log stream	Live tail + Select picker	Log viewer per container atau per pod

— Akhir Dokumen —